LaTeX for Research Students

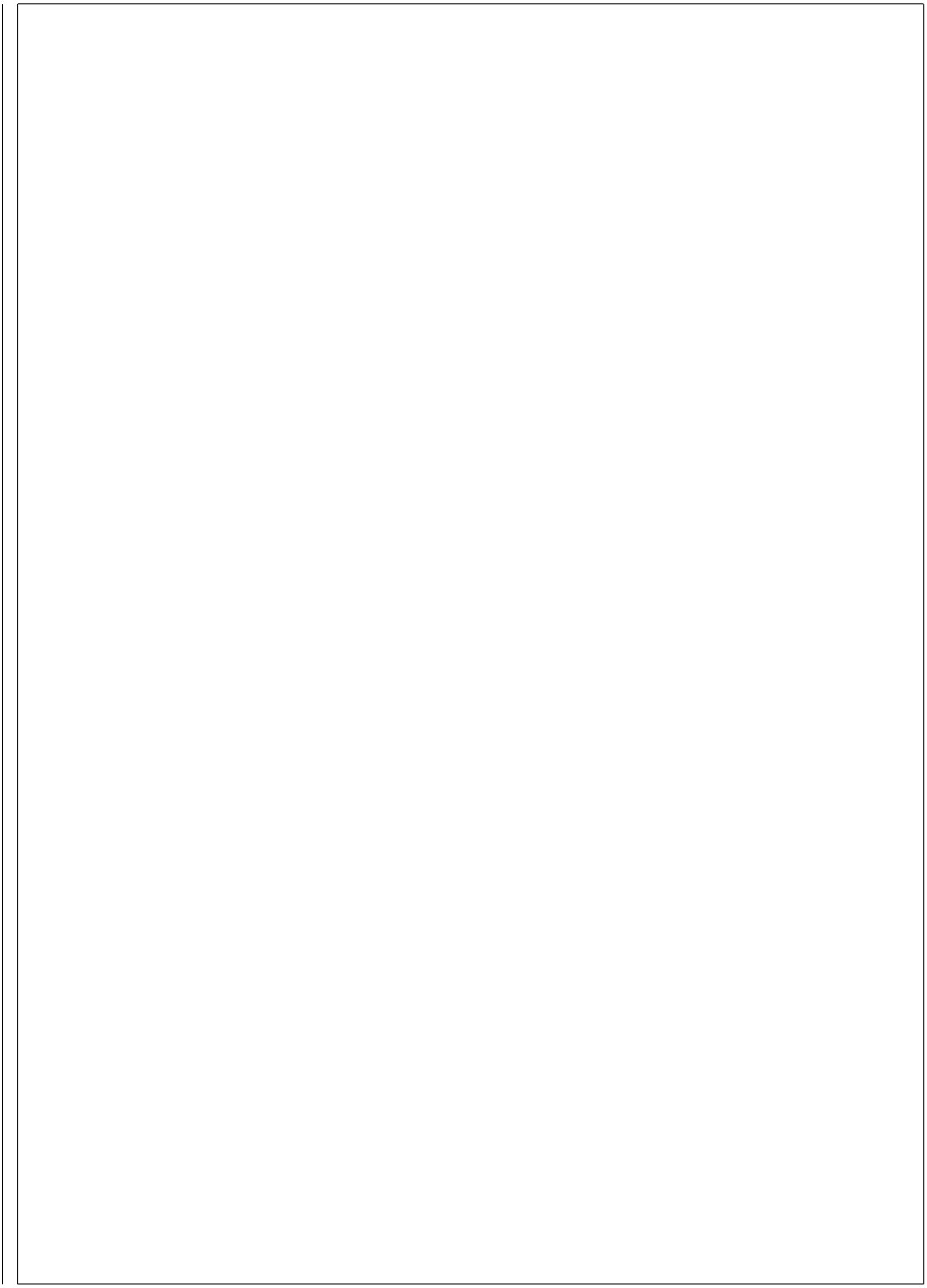
A beginner's guide with examples, nuances & exercises

Syed Ali Mohsin Bukhari [1, 2] and Iqra Siddique [3, 4, 5]

[1]Department of Applied Mathematics and Statistics, Institute of Space Technology, Islamabad 44000, Pakistan
[2]Space and Astrophysics Research Lab (SARL), National Centre of GIS and Space Applications (NCGSA), Islamabad 44000, Pakistan
[3]Gran Sasso Science Institute (GSSI), Viale Crispi 7, I-67100 L'Aquila, Italy
[4]Dipartimento di Fisica, Università di Trento, Via Sommarive 14 I-38123 Trento, Italy
[5]INFN Laboratori Nazionali del Gran Sasso, Via Acitelli 22, I-67100 Assergi, L'Aquila, Italy

July 26, 2026

Drafting assistant: Claude (Anthropic) · Reviewer: DeepSeek (High-Flyer)



Contents

LaTeX for Research Students: What it is? 1

Part 1: Foundations & Capstone Kickoff (Chapters 1–4) 3

Chapter 1 — Why LaTeX for Research Writing 5
What LaTeX is 5
Why researchers reach for it 6
When Word might still be fine 6

Chapter 2 — Setting Up Overleaf & Your First Document 7
Getting set up 7
Choosing a Compiler 8
Stopping at the First Error 8

Chapter 3 — Anatomy of a .tex File and Survival Debugging 9
The two zones of every document 9
Document Class & Package Options 10
Character Encoding with `inputenc` 10
Survival Basics: reading the log without panicking 11

Chapter 4 — Capstone Kickoff: Choosing Your Topic & Project Setup 13
The Capstone Project 13
Step 1 — Pick a topic 13
Step 2 — Set up the project shell 14

Part 2: Core Typesetting (Chapters 5–8) 15

Chapter 5 — Text Formatting & Sectioning 17
Inline Formatting & Sectioning 17
Line Breaks, Paragraphs & Non-Breaking Spaces 18
Sentence-Ending Spacing 18
Starred Variants (`*`) 19
Controlling TOC Depth & Heading Numbering 20
Line Spacing with `setspace` 20

Chapter 6 — Lists & Tables 23

Lists: `itemize` & `enumerate` . . . 23
Customizing List Labels with `enumitem` . . . 24
Professional Tables with `booktabs` . . . 25
Multi-Row & Multi-Column Cells . . . 26
Wide Tables: `sidewaystable` & `landscape` . . . 26
Aligned Decimal Columns with `siunitx` . . . 28
Other Table Tricks . . . 29

Chapter 7 — Figures & Floats **31**
Including a Basic Figure with `\includegraphics` . . . 31
Float Placement Specifiers . . . 32
Controlling Where Floats Actually Land . . . 32
Vector vs. Raster Images . . . 33
Handling Legacy `.eps` Figures . . . 33
Multi-Panel Figures with `subcaption` . . . 34

Chapter 8 — Cross-Referencing, Labels & Hyperlinks **37**
Automatic Numbering with `\ref` and `\cref` . . . 37
Styling Hyperlinks with `\hypersetup` . . . 38
The Labeling-Prefix Convention . . . 39

Part 3: Math Mechanics (Chapters 9–10) 41

Chapter 9 — Inline & display math basics/fonts & units **43**
Inline vs. display math . . . 43
Greek letters and sub/superscripts . . . 44
Italics vs. Roman in Math . . . 45
Formatting Units . . . 45
Bold Vectors & Greek Letters with `bm` . . . 47
Putting It Together . . . 47

Chapter 10 — Advanced Math: Matrices, Multi-line Equations, Cases **49**
Matrices . . . 49
Piecewise Functions with `cases` . . . 50
Multi-Step Derivations . . . 50
Controlling Fraction Size . . . 51
Delimiter Sizing . . . 51
One-Sided Delimiters . . . 52
Math Spacing . . . 52

Appendix 55

A.1 — Symbol & Command Cheat Sheet **59**

A.2 — Table Formatting Cheat Sheet **61**
`minipage` — and how it differs from `subfigure` . . . 62

A.3 — Page Layout & Spacing Reference **65**

A.4 — Common Package Reference Table **67**

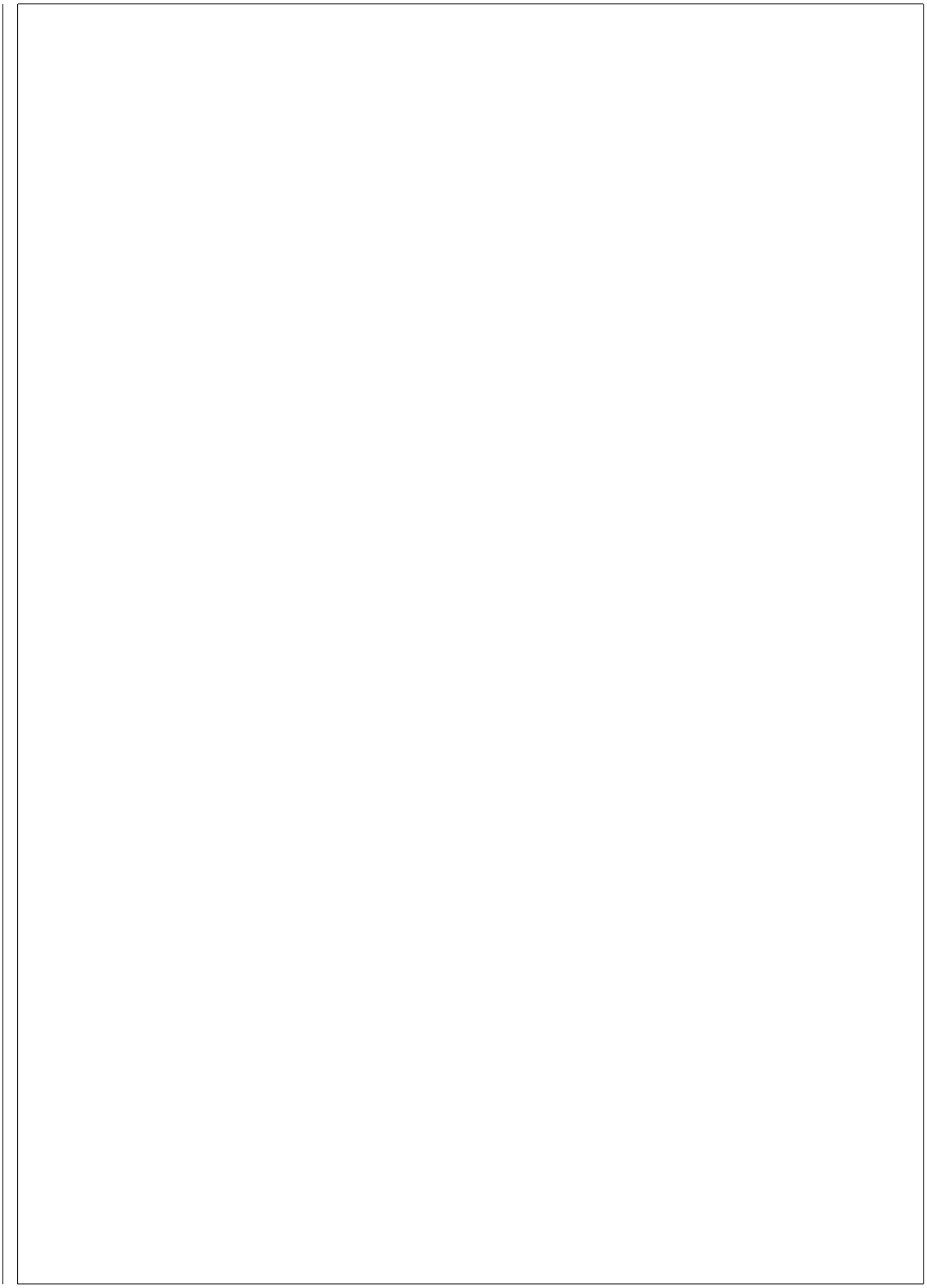
A.5 — Compiler Comparison **71**

A.6 — `tikz-cd` and `chemfig` Starter Snippets **73**

A.7 — Quick Troubleshooting Checklist **75**

A.8 — Starter `.bib` Files **77**

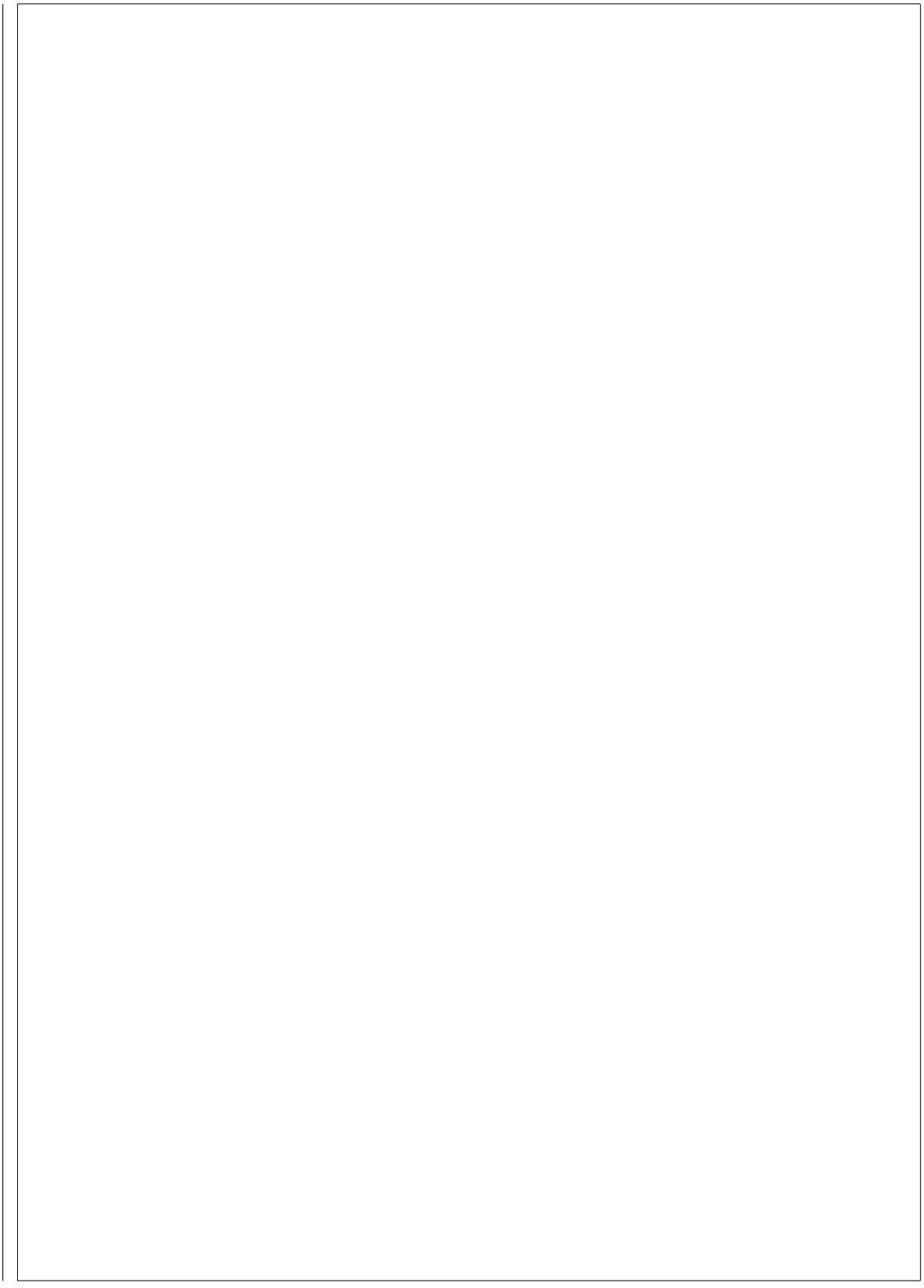
A.9 — `hypersetup` Snippet **79**



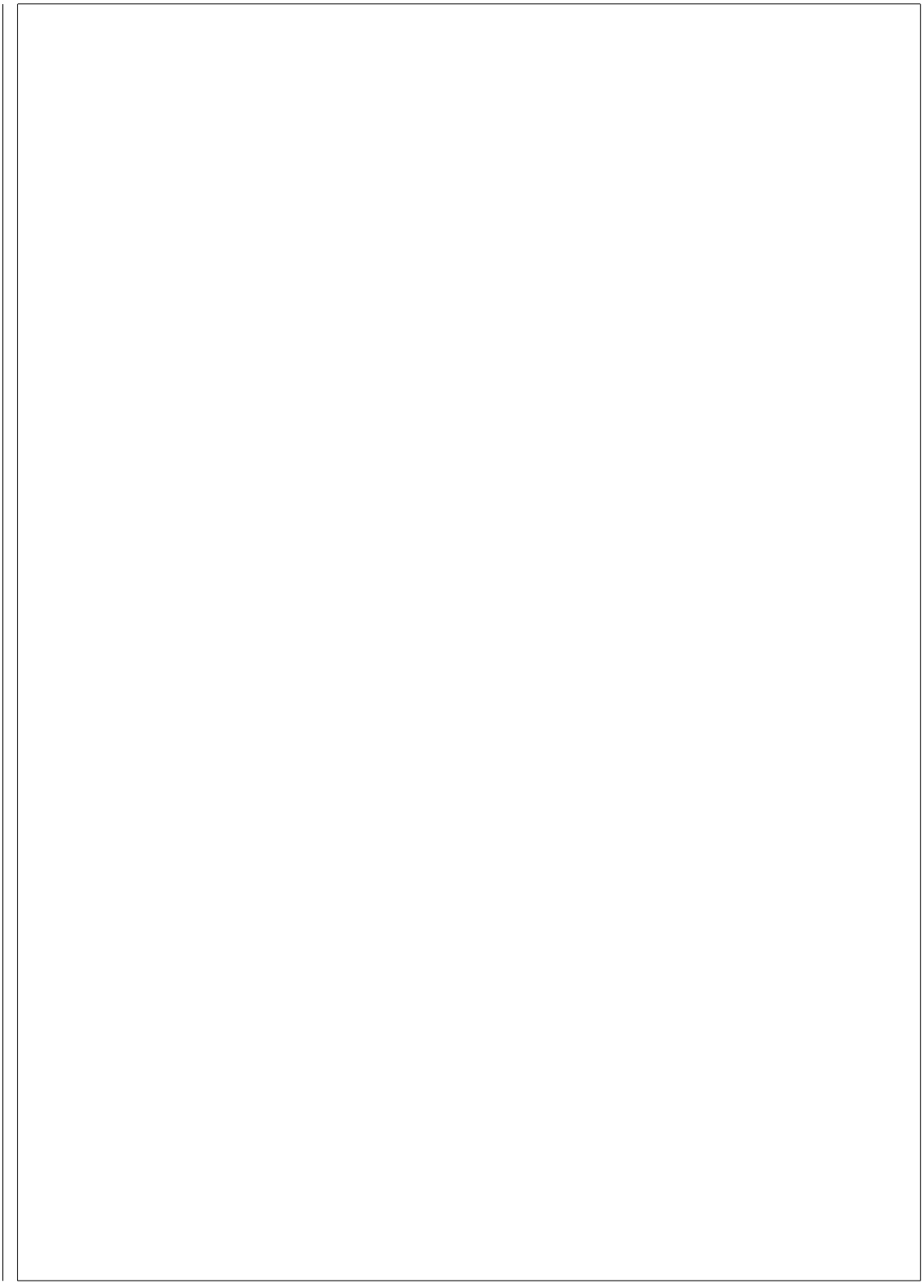
LaTeX for Research Students: What it is?

This guide takes you from your first compiled document to a complete, submission-ready research paper. It's written for students in STEM and other research fields who are new to LaTeX, and it assumes no prior experience — just a willingness to work through the examples and exercises as you go. A running capstone project threads through the chapters: starting in Chapter 4 you build one paper piece by piece, so that by the end you have both the knowledge and a finished document to show for it.

The material is organized into nine Parts (Chapters 1–23), followed by an Appendix of reference material and a Supplementary Materials file holding larger code samples and datasets referenced from the chapters.



Part 1: Foundations & Capstone Kickoff (Chapters 1–4)



Chapter 1 — Why LaTeX for Research Writing

Every research document eventually raises the same question: why fight with markup and a compiler instead of just clicking bold in a word processor? It doesn't pay off for everything — but for the kind of writing this guide is aimed at, it pays off often enough that most STEM researchers end up using it sooner or later.

This chapter covers what LaTeX actually is, the concrete reasons researchers reach for it over Word, and — just as importantly — the cases where it isn't the right tool, so the choice to use it is deliberate rather than automatic.

Objective 1

Understand what LaTeX actually is, why research papers and theses are so often written in it, and when it's the right tool for the job.

What LaTeX is

LaTeX is a *typesetting system*, not a word processor. Instead of clicking buttons to make text bold or center a heading, you write plain text with markup commands, and a compiler turns that markup into a formatted PDF. A Word document is WYSIWYG ("what you see is what you get"); a LaTeX document is closer to "what you *describe* is what you get."

Worked Example 1.1

```
\documentclass{article}

\begin{document}
    Hello, research world.
\end{document}
```

Nuance 1.1

Compiling this produces a properly typeset PDF page — margins, fonts, and spacing all handled by LaTeX's rules, not by you nudging things around manually.

Why researchers reach for it

A handful of concrete advantages come up again and again once you've used it on a real project:

- **Math typesets beautifully and consistently.** An equation with nested fractions, summations, or matrices is a few lines of markup instead of a fight with an equation editor.
- **Long documents stay consistent.** Change the font size of every section heading in a 200-page thesis with one line, not 40 manual edits.
- **Bibliographies are managed, not typed.** Citations and reference lists are generated from a database file — add a source once, cite it anywhere (Part IV covers this in depth).
- **It's the default for most STEM journals.** Many journals provide official LaTeX templates (`.cls` files) and *prefer* or *require* LaTeX submissions.
- **Plain text plays well with version control.** A `.tex` file is just text, so tools like Git track meaningful changes (Ch. 21 picks this up).

When Word might still be fine

LaTeX has a learning curve. For a one-page memo, a quick letter, or a document with no math and no references, Word (or Google Docs) will get you there faster. LaTeX pays off as documents get longer, more mathematical, and more collaborative — which describes most research writing, but not all writing.

(Context chapter — no formal exercise. If you want a first taste, try compiling the snippet above once you reach Ch. 2.)

Chapter 2 — Setting Up Overleaf & Your First Document

Chapter 1 made the case for LaTeX in the abstract; this one gets an actual compiled PDF in front of you, which matters more than any argument for why the tool is worth learning.

This chapter covers creating an Overleaf account and project, compiling your first document, and two settings — compiler choice and "stop on first error" — that are easy to skip past but save real time once documents get more complex.

Objective 2

Create an Overleaf account, start a project, understand the compile button, and produce your first compiled PDF.

Getting set up

1. Go to overleaf.com and create a free account (a university email often unlocks extra features).
2. Click **New Project** → **Blank Project**. This opens an editor pane (your `.tex` code) next to a PDF preview pane.
3. Click the green **Recompile** button. Overleaf runs the LaTeX compiler and renders the PDF on the right.

No local software install is required — this is why Overleaf is the recommended starting point for this guide. Replace the default project content with:

Worked Example 2.1

```
\documentclass{article}

\title{My First LaTeX Document}
\author{Your Name}
\date{\today}

\begin{document}

    \maketitle
```

```
    \section{Introduction}
    This is my first properly structured LaTeX document.
    It has a title, an author, a date, and a numbered section.

\end{document}
```

Recompile, and you should see a title block (title, author, today's date) followed by a numbered section. The `\maketitle` command is what actually renders the title/author/date you declared above it — forgetting this call is a common first mistake (nothing shows up even though you set a title).

Choosing a Compiler

Overleaf's compiler choice quietly decides which packages even work, which is worth knowing before a font package mysteriously fails to compile. It lets you choose the compiler under the project's **Menu** → **Compiler**. This matters more than it looks:

Important 2.1

If you use the `fontspec` package to change fonts, you *must* compile with `XeLaTeX` or `LuaLaTeX` — `pdfLaTeX` can't process it. If you don't need custom fonts, stick with `pdfLaTeX`; most journals require it for submission, and most packages assume it by default.

Stopping at the First Error

Left on Overleaf's default "Always compile," a single missing `$` can cascade into fifty confusing follow-on errors that all trace back to one broken line.

Important 2.2

Open the **Recompile** dropdown (small arrow next to the button) and check **"Stop on first error."** Stopping at the first error tells you exactly which line broke — turn this on now, you'll thank yourself in Ch. 3. The error LaTeX reports will likely point to the line *after* the missing brace (e.g., your `\author{...}` line), not the `\title{...}` line that's actually broken. This is normal — LaTeX doesn't realize a brace is unclosed until it hits the *next* command. When you see a strange error, always check the line above first.

Exercise 2.1

Create and compile a title page for a fictional paper:

1. Set a `\title{}`, two `\author{}` names separated by `\and`, and `\date{}`.
2. Add one section with two sentences of placeholder text.
3. Confirm "Stop on first error" is enabled, then deliberately delete the closing `}` on your title, and recompile — read the error message before fixing it.

Chapter 3 — Anatomy of a .tex File and Survival Debugging

Every `.tex` file you'll ever open — yours or someone else's — is built from the same two zones and the same handful of moving parts, so recognizing them on sight is what turns an unfamiliar file from intimidating into just another document.

This chapter covers that anatomy (preamble vs. body, document class, packages, options), plus the minimum debugging habits — reading the compile log, and compiling often — needed to survive your first real errors without panicking.

Objective 3

Understand the parts of a `.tex` file (preamble vs. body, document class, packages) well enough to read *any* LaTeX file, and learn the minimum debugging skills to survive your first errors.

The two zones of every document

Worked Example 3.1

```
\documentclass[12pt]{article}   % <- PREAMBLE starts here
\usepackage{amsmath}            %    packages, settings, custom commands
\usepackage{graphicx}           %    (nothing here is "printed")

\begin{document}                % <- BODY starts here
This text appears in the PDF.   %    everything you write here is rendered
\end{document}                  % <- document ends
```

- **Preamble** (everything before `\begin{document}`): declares the document class, loads packages, and configures global settings. Nothing here appears in the output.
- **Body** (between `\begin{document}` and `\end{document}`): the actual content that gets typeset.
- **Document class** (`\documentclass{article}`): sets the overall document type (`article`, `report`, `book`, or a university/journal-specific class). It determines what section levels exist and how they're numbered.

- **Packages** (`\usepackage{...}`): each one adds a capability — `amsmath` for advanced math, `graphicx` for images, and so on. You'll accumulate a standard set of these as you go through this guide.

Document Class & Package Options

Both `\documentclass` and `\usepackage` accept a bracketed list of options that tweak behavior without changing the class or package name itself — worth recognizing as one consistent syntax rather than memorizing each occurrence separately.

Nuance 3.1

The `[12pt]` font size in `\documentclass[12pt]{article}` above, and the `[utf8]` option in `\usepackage[utf8]{inputenc}` later in this chapter, are both **options** — a comma-separated list in square brackets that configures how the class or package behaves, separate from the class/package name itself.

General 3.1 — 'documentclass' options

A few common `\documentclass` options worth knowing by name, even before you need them:

- **Font size**
  - `10pt` (the default if you omit this), `11pt`, `12pt` — sets the base text size for the whole document.
- `twoside/oneside`
  - Whether the document is meant for double-sided printing, which affects margins (alternating sides for binding) and page-parity behavior (Ch. 7's `\cleardoublepage`).
  - `article` and `report` default to `oneside`; only `book` defaults to `twoside`.
- `twocolumn`
  - Starts the entire document in two-column layout from the first page.
  - Ch. 20 covers switching mid-document with `\onecolumn`/`\twocolumn` instead, but if the *whole* document needs two columns, as most two-column journal templates do, setting it here as a class option is simpler than switching manually.
- `draft`
  - Speeds up compiling by replacing images with outlined boxes and flagging overfull lines with a black bar in the margin.
  - Useful while iterating on a long document; remove it before your final compile.

Multiple options are comma-separated: `\documentclass[12pt,twoside,draft]{report}`.

Character Encoding with `inputenc`

Typing an accented letter or a pasted "smart quote" directly into a pdfLaTeX source file needs one extra declaration before the compiler interprets it correctly instead of choking on it.

`\usepackage[utf8]{inputenc}` declares that your source `.tex` file is written in UTF-8 — this is what lets pdfLaTeX correctly interpret non-ASCII characters typed directly into the file (accented letters, "smart quotes" pasted from elsewhere, and so on). It's specific to pdfLaTeX: XeLaTeX

and LuaLaTeX (Ch. 2's compiler nuance) are UTF-8-native by default and don't need it — loading `inputenc` alongside them is harmless, just redundant.

Survival Basics: reading the log without panicking

When a document compile fails, Overleaf highlights the offending line and shows a message in the log panel below the editor. The two most common first errors:

Message	Usual cause
`Undefined control sequence`	You typo'd a command name, or forgot to load the package that defines it
`Missing $ inserted`	You used a math-only character (like `_` or `^`) outside math mode, or left a `$` unclosed

The habit that saves the most time: compile *often* — after every few lines, not after writing a whole page. If something breaks, you know it's in the last few lines you added, not buried somewhere in 200 lines of text. Combined with "Stop on first error" from Ch. 2, this turns debugging from a hunt into a quick glance.

(This is deliberately the *minimum* survival kit. Ch. 22 returns to debugging in full depth once you've hit enough real errors to appreciate it.)

Worked Example 3.2

```
\documentclass{article}
\usepackage[utf8]{inputenc}
\usepackage{graphicx}

\title{Anatomy of a LaTeX File}
\author{Your Name}

\begin{document}
    \maketitle

    \section{Preamble vs Body}
    Everything above this document's begin-document tag is configuration.
    Everything below it is content.

\end{document}
```

This snippet compiles cleanly as-is. The exercise below will have you introduce a deliberate error yourself, so you get practice diagnosing one rather than hunting for a phantom bug.

Exercise 3.1

1. Build the document skeleton above from scratch (don't copy-paste) so the structure sticks.
2. Deliberately break it: remove the `\usepackage{graphicx}` line, then add

`\includegraphics{}` anywhere in the body. Recompile and read the resulting `Undefined control sequence` error — this is exactly what it looks like when you use a command without loading the package that defines it.

3. Fix it (restore the `\usepackage{graphicx}` line), recompile, and confirm the PDF renders cleanly.

Chapter 4 — Capstone Kickoff: Choosing Your Topic & Project Setup

The first three chapters covered LaTeX in the abstract — what it is, how to compile it, what a file is made of. This chapter is where that knowledge turns into an actual, evolving document: you pick a topic and stand up a minimal project shell, and every chapter from here through Ch. 23 adds one more piece to it.

Objective 4

Choose a capstone topic and set up the minimal, compiling project shell that every later chapter's Capstone Update will build onto.

The Capstone Project

Starting now, every remaining chapter (5 through 22) ends with a short **Capstone Update** — you apply that chapter's lesson directly to one evolving paper. By Ch. 23, you'll have assembled a complete, submission-formatted document built piece by piece.

Step 1 — Pick a topic

Choose whichever of these four fits your field (or is closest to your own research — feel free to substitute your real topic once you're comfortable):

1. **Physics:** Thermal conductivity of graphene composites.
2. **CS/Math:** Convergence analysis of a gradient descent algorithm.
3. **Engineering:** Stress-strain analysis of a cantilever beam.
4. **Biology/Chem:** Reaction rate of enzyme kinetics.

Starter `.bib` files (5 dummy references each) for all four topics are provided in the guide's supplementary materials, so you'll have real citations to work with from Part IV onward regardless of which you pick.

Step 2 — Set up the project shell

Create a fresh Overleaf project and start it with a minimal, honest skeleton — no content yet, just the scaffolding every later chapter will build onto:

Worked Example 4.1

```
\documentclass[12pt]{article}
\usepackage[utf8]{inputenc}

\title{[Working Title - will be refined in Ch. 5]}
\author{Your Name}
\date{\today}

\begin{document}
    \maketitle

    \begin{abstract}
        [Placeholder: one-sentence summary of your chosen topic.
        Will be expanded as later chapters add content.]
    \end{abstract}

    \section{Introduction}
    % Content added starting in Ch. 5

\end{document}
```

Compile it once now to confirm the shell is error-free before building on it.

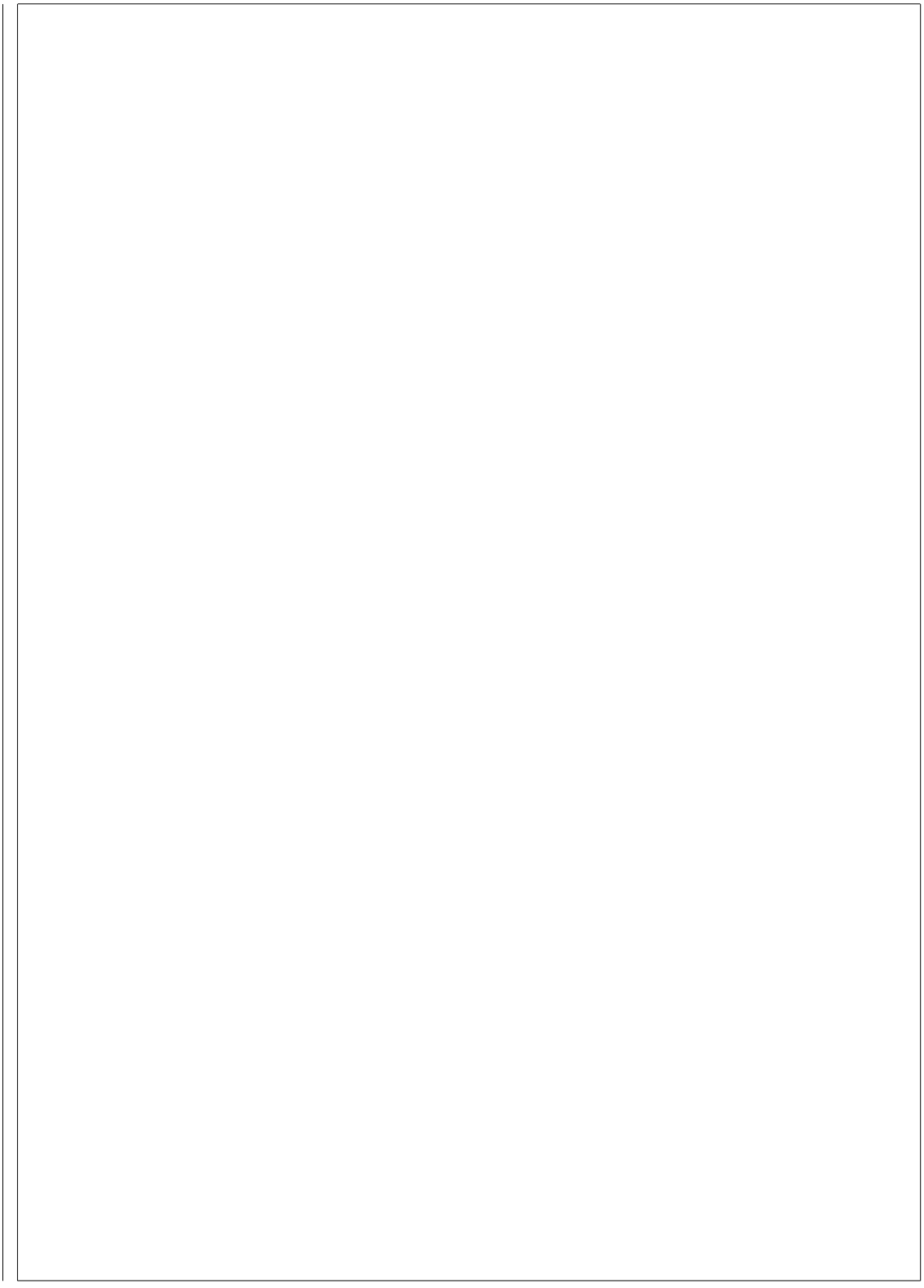
Don't worry if the abstract wording isn't perfect yet — it's a placeholder. You'll rewrite it properly once later chapters give you actual content (results, methods) to summarize.

Capstone Update 4

Create your project shell:

- Pick one of the four topics (or your own, if you're confident enough to skip the training wheels).
- Set the title, author, date, and a one-line abstract placeholder as shown above.
- Compile successfully — this file is what every following chapter's Capstone Update will extend.

Part 2: Core Typesetting (Chapters 5–8)



Chapter 5 — Text Formatting & Sectioning

Every document beyond a single page needs two things sorted out early: basic inline emphasis (bold, italic — for terms, warnings, foreign phrases) and a heading hierarchy that actually holds a long document together. Getting the second one wrong is what causes the most confusion later — a thesis with inconsistent section numbering, or a table of contents that silently drops a heading level, is almost always a symptom of never having set `tocdepth`/`secnumdepth` explicitly, or of nesting sections one level deeper than the document class expects.

This chapter covers both: `\section`/`\subsection`/`\subsubsection` for structured formatting, `\textbf`/`\textit` and friends for inline formatting, and `\tableofcontents` to generate a TOC from that structure automatically. It also covers two habits — starred (unnumbered) variants, and `setspace` for draft line-spacing — that show up constantly from here on, so it's worth having them settled before the rest of the guide starts assuming you know them.

Objective 5

Format text (bold, italic), structure a document with sections/subsections, and generate a working table of contents.

Inline Formatting & Sectioning

Every document needs the same two building blocks early on: a way to emphasize individual words inline, and a heading hierarchy — `\section` down to `\subsubsection` — that `\tableofcontents` can turn into a working table of contents.

Worked Example 5.1

```
\documentclass{article}
\usepackage[utf8]{inputenc}

\title{Formatting \& Sectioning Basics}
\author{Your Name}

\begin{document}
    \maketitle
```

```
    \tableofcontents

    \section{Introduction}
    This is \textbf{bold} text and this is \textit{italic} text.
    You can also combine them: \textbf{\textit{bold italic}}.

    \subsection{Background}
    A subsection nested under the introduction.

    \subsubsection{Fine Detail}
    A subsubsection nested under the subsection.

\end{document}
```

Nuance 5.1

Recompile twice if the table of contents looks empty or stale on the first try — LaTeX writes the TOC to an auxiliary file during compilation and reads it back in on the *next* compile. This is normal, not a bug.

Line Breaks, Paragraphs & Non-Breaking Spaces

LaTeX treats a line break, a new paragraph, and a plain space as three separate things. Mixing them up is what produces a paragraph with no indent, or a line that breaks in an ugly spot.

Nuance 5.2

Three small habits worth getting right early, since you'll use all three constantly from here on:

- `\\` forces a line break without starting a new paragraph (no first-line indent) — you'll see it inside `align`/`gather` environments in later chapters.
- `\newline` behaves the same way in most contexts, but isn't universally interchangeable with `\\` — inside a `tabular` row, for instance, only `\\` works.
- A genuine new paragraph comes from a **blank line** in your source (or, equivalently, an explicit `\par`) — that's what actually gives you the indentation and spacing that "looks like" a new paragraph. `\\` alone does not do this; it only breaks the line.
- `~` inserts a non-breaking space — one LaTeX will never break a line on. Use it to keep short, related pieces of text glued together, like `Figure~5` or `Dr.~Smith`, so a line break can't land awkwardly between them. You'll use this constantly starting in Ch. 8, once `\cref`/`\ref` are introduced — `Figure~\cref{fig:...}` is the standard pairing.

Sentence-Ending Spacing

LaTeX tries to guess whether a period ends a sentence purely from the letter case right before it, and that heuristic breaks down around abbreviations and initials — worth knowing so the fix

doesn't feel like a mystery the first time you need it.

Nuance 5.3

LaTeX inserts extra space after what it assumes is the end of a sentence, and it guesses based on the character right before the period:

- A period after a **lowercase** letter is assumed to end a sentence, and gets the wider spacing by default — correct most of the time, but wrong for a lowercase abbreviation used mid-sentence, like "e.g." or "i.e.". Fix it with a backslash-space or `~` right after the period: `e.g.\ apples` or `e.g.~apples`.
- A period after an **uppercase** letter is assumed to be an abbreviation (an initial, or something like "NASA"), and gets the narrower spacing by default — wrong when that period genuinely ends the sentence. Fix it with `\@` right before the period: `NASA\@. We went...`

If you'd rather not think about this distinction at all, `\frenchspacing` in your preamble disables it entirely and uses the same spacing everywhere — a reasonable default some style guides prefer.

Starred Variants (*)

A pattern shows up here for the first time that recurs constantly for the rest of the guide — appending `*` to a command or environment name to turn off its automatic numbering — so it's worth learning as a general reflex rather than a one-off trick for `\section`.

Nuance 5.4

Many LaTeX commands and environments have a **starred variant** — the same name with a `*` appended — which almost always means "the same thing, but unnumbered." The most common first encounter is an unnumbered section, useful for things like an "Acknowledgments" heading that shouldn't get a number:

```
\section{Introduction}       % numbered: "1  Introduction"
\section*{Acknowledgments}   % unnumbered: just "Acknowledgments"
```

A rule of thumb is, whenever you see a trailing `*` on a LaTeX command or environment, assume "same behavior, minus the automatic numbering" and check the specific case for any extra detail.

Important 5.1

`\section*{}` also doesn't appear in the table of contents by default (since the TOC is built from numbered headings). If you want it listed anyway, add this line right after the `\section*{}` call:

```
\addcontentsline{toc}{section}{Acknowledgments}
```

Cross-Check 5.1 — Cross-Check: The star-pattern cheat sheet

This same star pattern reappears throughout the guide, so it's worth recognizing on sight rather than re-learning each time:

- `equation*` (Ch. 9) — a display equation with no number.
- `align*` (Ch. 10) — multiple aligned equations, none numbered.
- `figure*`/`table*` — in two-column document classes (common in some journal templates), a float that spans both columns instead of sitting in just one.

Controlling TOC Depth & Heading Numbering

The document class quietly caps how deep the table of contents and heading numbers go, so a `\subsubsection` can look "wrong" for reasons that have nothing to do with how it was typed — two counters in the preamble are what actually control that depth.

Nuance 5.5

By default, `\subsubsection` and `\paragraph` headings can silently disappear from the TOC, or appear unnumbered, which causes panic the first time it happens. Control both behaviors explicitly in your preamble:

```
\setcounter{tocdepth}{3} % include subsubsections in the TOC
\setcounter{secnumdepth}{3} % number them too
```

Important 5.2

Heading depth in `article`:

1. `\section` = 1,
2. `\subsection` = 2,
3. `\subsubsection` = 3,
4. `\paragraph` = 4.

Many universities leave `\paragraph` unnumbered by default — these two counters are how you take control of that instead of guessing why a heading looks "wrong."

Line Spacing with `setspace`

Draft and thesis formatting requirements frequently specify double or 1.5 line spacing, and hand-rolling that with `\vspace` or `\linespread` throws off table and math spacing elsewhere in the document — a dedicated package avoids the side effects.

Nuance 5.6

Thesis offices often require double or 1.5 line spacing for draft submissions; journals usually want single spacing. Don't reach for `\vspace` or `\linespread{2}` by hand — it distorts table and math spacing. Use the `setspace` package instead:

```
\documentclass{article}

\usepackage{setspace}

\begin{document}
    \doublespacing
    % ... rest of the document is now double-spaced ...

    \begin{singlespace}
        % a dense table or the abstract,
        % kept readable even in an otherwise double-spaced document
    \end{singlespace}

\end{document}
```

Nuance 5.7

`\onehalfspacing` and `\singlespacing` work the same way if you need those instead.

Cross-Check 5.2 — Cross-Check

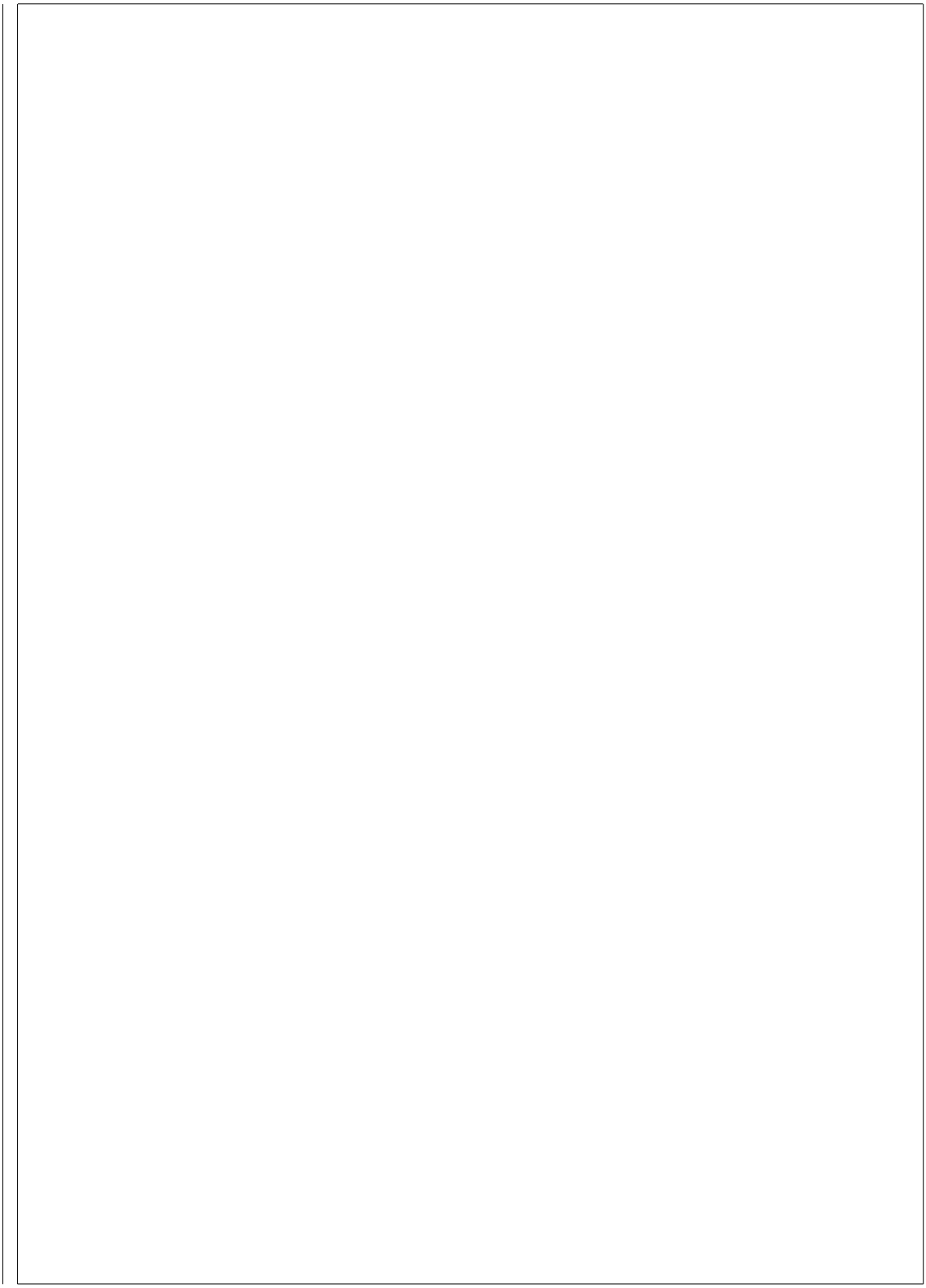
Want to see the effects for yourself, with a working example, rather than just read about it? `supplementary_materials.md` has a ready-made dense table you can paste in and toggle `singlespace` on/off around — see "Ch. 5 — Dense Table for Testing `setspace`."

Exercise 5.1

1. Take your Ch. 4 capstone shell and add two more section levels (`\subsection`, `\subsubsection`) somewhere in the introduction.
2. Add `\tableofcontents` after `\maketitle` and recompile twice to see it populate.
3. Set `tocdepth`/`secnumdepth` to 3 and confirm your subsubsection now appears, numbered, in the TOC.

Capstone Update 5

- Draft real section headings for your paper (e.g. `Introduction`, `Methods`, `Results`, `Discussion`, `Conclusion`) instead of the single placeholder `\section{Introduction}` from Ch. 4.
- Decide whether your target format needs double-spacing (thesis draft) or single (journal submission), and set it with `setspace`.
- Confirm the TOC shows the depth you want.



Chapter 6 — Lists & Tables

Research writing constantly needs to present information as discrete points or structured data rather than flowing prose — a list of assumptions, a sequence of experimental steps, a table of measurements. LaTeX handles both natively, but its *default* output for each is noticeably plain: bullets/numbers with no styling control, and tables ringed with the boxy vertical/horizontal rules typical of a spreadsheet rather than a printed page. Neither is wrong, exactly, but neither is what you'll see in a published paper or a well-typeset thesis either.

This chapter starts from those defaults and builds up to what journals and thesis offices actually expect: customizable list labels (lettered, Roman numeral, tightly spaced), and tables built with `booktabs`'s minimal open rules and `siunitx`'s decimal-aligned numeric columns — plus fixes for the two problems wide tables run into (columns overflowing the page, and numbers that don't line up).

Objective 6

Format lists and build clean, professional-looking tables — including wide tables and tables with aligned numeric data.

Lists: `itemize` & `enumerate`

LaTeX's two built-in list environments cover the two everyday cases out of the box: `itemize` for unordered/bulleted points, `enumerate` for numbered steps.

Worked Example 6.1

```
\begin{itemize}
    \item First point
    \item Second point
    \item Third point
\end{itemize}

\begin{enumerate}
    \item Step one
    \item Step two
    \item Step three
\end{enumerate}
```

Nuance 6.1

The `itemize` environment gives bullets; `enumerate` environment gives numbers. Both can be nested inside each other.

Customizing List Labels with `enumitem`

Default bullets and numbers cover most cases, but journal or thesis styles sometimes want something different — lettered sub-items, Roman numerals, or a tighter list in a dense layout.

Nuance 6.2

The `enumitem` package handles all of it with one consistent syntax:

```
\documentclass{article}
\usepackage{enumitem}

\begin{document}

    \begin{enumerate}[label=(\alph*)]
     \item First
     \item Second
    \end{enumerate}
    % renders as (a), (b), ...

    \begin{enumerate}[label=\Roman*.]
     \item First
     \item Second
    \end{enumerate}
    % renders as I., II., ...

    \begin{itemize}[nosep, leftmargin=*]
     \item Tight, flush-left bullet
     \item No vertical space between items
    \end{itemize}

\end{document}
```

General 6.1 — Usage

- `label=`
  - Controls what marks each item — `\alph*`/`\Alph*` for lower/uppercase letters, `\roman*`/`\Roman*` for lower/uppercase Roman numerals, alongside the default `\arabic*`.
- `nosep`
  - Strips the vertical spacing between items, useful for a dense list inside a table cell or a tight layout.
- `leftmargin=*`
  - Flushes the list to the left margin instead of the default indent — common in some thesis styles that want lists aligned with body text.

Professional Tables with `booktabs`

A plain `tabular` with `\hline` and vertical bars (`|c|c|`) technically works, but reads as dated compared to the open, rule-light tables standard in published papers — `booktabs` is the package that gets you there:

Worked Example 6.2

```
\documentclass{article}
\usepackage{booktabs}

\begin{document}

    \begin{table}[htbp]
        \centering
        \caption{Sample thermal conductivity measurements}
        \label{tab:sample}
        \begin{tabular}{lcc}
            \toprule
            Material   & Conductivity (W/mK) & Temperature (K) \\
            \midrule
            Graphene A & 5000                & 300             \\
            Graphene B & 4200                & 350             \\
            \bottomrule
        \end{tabular}
    \end{table}

\end{document}
```

Nuance 6.3

`booktabs` gives you `\toprule`, `\midrule`, and `\bottomrule` instead of `\hline` — no vertical rules at all, which is the field standard for professional tables. A cheat sheet comparing the two styles is in the appendix.

Multi-Row & Multi-Column Cells

A table cell sometimes needs to span more than one row or column instead of holding a single value — a group header that sits above two sub-columns ("Measurement" spanning "Trial 1" and "Trial 2"), or a sample name that applies to several data rows underneath it and shouldn't be repeated on every line. Plain `tabular` has no way to do either; two commands handle it, one column-wise and one row-wise:

Worked Example 6.3

```
\documentclass{article}
\usepackage{multirow}

\begin{document}
    \begin{tabular}{lcc}
        \toprule
        \multirow{2}{*}{Sample}   & \multicolumn{2}{c}{Measurement}   \\
        & Trial 1 & Trial 2                  \\
        \midrule
        A                         & 10      & 12                      \\
        \bottomrule
    \end{tabular}
\end{document}
```

Nuance 6.4

- `\multicolumn{cols}{align}{text}` is built into LaTeX already.
- `\multirow{rows}{width}{text}` (from the `multirow` package) is the one that needs an explicit `\usepackage`.

Wide Tables: `sidewaystable` & `landscape`

A table with enough columns will overflow the page margins regardless of how well it's styled, and two packages fix that by rotating things 90 degrees — they differ only in *how much* they rotate.

sidewaystable

Worked Example 6.4

```
\documentclass{article}

\usepackage{rotating}

\begin{document}
    \begin{sidewaystable}
        \centering
        \caption{A wide table of supplementary data}
        \begin{tabular}{cccccccc}
            \toprule
            ... 8 columns of data ...
            \bottomrule
        \end{tabular}
    \end{sidewaystable}
\end{document}
```

Nuance 6.5

The one-line fix is the `rotating` package's `sidewaystable` environment — it rotates the entire table (and its caption) 90 degrees. Great for supplementary-materials tables that just don't fit portrait orientation.

landscape

Worked Example 6.5

```
\documentclass{article}
\usepackage{pdflscape}

\begin{document}
    \begin{landscape}
        \begin{table}[htbp]
            \centering
            \caption{An especially wide table, needing a full landscape page}
            \begin{tabular}{cccccccccc}
                \toprule
                ... 10 columns of data ...
                \bottomrule
            \end{tabular}
        \end{table}
    \end{landscape}
\end{document}
```

Nuance 6.6

`sidewaystable` above rotates a single float on an otherwise-portrait page. Sometimes that's still not enough — a table wide enough that even a rotated float feels cramped, or you want a full landscape page with its own header and margins. That's a job for the `landscape` environment instead, from `lscape` (or `pdflscape` — the pdfLaTeX-safe version; use this one on Overleaf's default compiler, since it also sets the PDF's page-rotation metadata so viewers display it right-side-up).

Important 6.1

1. `sidewaystable` rotates a float within an otherwise-portrait page; `landscape` rotates the physical page itself.
2. Reach for `sidewaystable` first — it's simpler and keeps everything else on the page portrait — and only switch to `landscape` when a single rotated float genuinely isn't wide enough.

Aligned Decimal Columns with `siunitx`

Numeric results (regression coefficients, experimental measurements) when laid out in a plain `c` column are not eye friendly, or eye catching, since the decimal points don't align down the column. It also becomes hard to scan and a bit unprofessional-looking.

Worked Example 6.6

```
\documentclass{article}
\usepackage{siunitx}

\begin{document}
    \begin{tabular}{l S[table-format=3.2]}
        \toprule
        {Sample} & {Conductivity (W/mK)} \\
        \midrule
        A        & 123.45                \\
        B        & 98.60                 \\
        \bottomrule
    \end{tabular}
\end{document}
```

Nuance 6.7

The `siunitx` package's `S` column type fixes this — declare the decimal precision once, and every row's decimal point lines up automatically.

Important 6.2

`S[table-format=3.2]` reserves 3 digits before and 2 after the decimal point, and aligns every row's decimal point vertically. Note the curly braces around non-numeric header text (`{Sample}`, `{Conductivity...}`) — the `S` column expects numbers by default, so text needs to be explicitly wrapped.

Other Table Tricks

General 6.2 — Table cheat sheet

The tools below solve real, specific table problems, but aren't everyday defaults — reach for them only when you actually hit the edge case each one is for:

- **`@{}` column specifier** — removes the automatic padding LaTeX inserts between columns, for columns that should visually touch: `\begin{tabular}{l @{} c}`.
- **`\resizebox{\textwidth}{!}{...}`** (from `graphicx`, Ch. 7) — scales an entire table to fit the page width. Use sparingly: it shrinks the font along with everything else, which can make a table technically fit but practically unreadable.
- **`tablefootnote`** — a regular `\footnote` breaks inside a `table` float (it either silently disappears or lands on the wrong page); `tablefootnote` fixes that.
- **`threeparttable`** — for a table that needs *several* notes tied to the table's own width rather than the page's — a more formal notes block underneath, versus `tablefootnote`'s one quick note at a time.
- **`minipage`** — places a table next to text, or next to another small table, side by side rather than stacked.
- Row height can also be adjusted, globally or per-table, with `\arraystretch` — covered in full in Ch. 19, once `\renewcommand` has been introduced.

Cross-Check 6.1 — Cross-Check

See `supplementary_materials.md` for a single table that puts several of these to work together — more useful to see combined than as isolated one-liners.

Exercise 6.1

1. Recreate this data as a table using `booktabs`: `Sample,Value` / `A,12.5` / `B,7.3` / `C,150.25`
2. Do it twice: once with a plain `c` column, once with an `S[table-format=3.2]` column — compare how the decimals line up.
3. Bonus: wrap the same table in `sidewaystable` and observe the orientation change.
4. Reformat the `enumerate` list from the very first worked example using `enumitem`: switch it to lettered labels `(a)`, `(b)`, `(c)`, and tighten it with `nosep`.

Nuance 6.8

When using the `S` column, wrap your column headers in curly braces, e.g. `{Value}`, so `siunitx` doesn't try to parse them as numbers and throw an "Invalid token" error.

Capstone Update 6

Add a result table to your paper using the topic's own data (real or plausible placeholder numbers), with `booktabs` for the rules and `S` columns for any numeric measurements. If your table ends up especially wide or needs a footnoted value, this is also a good place to try `sidewaystable`/`landscape` or `tablefootnote` for real instead of just in the exercise.

Chapter 7 — Figures & Floats

Almost every research document needs to include images — plots, diagrams, photographs of apparatus — and LaTeX treats them as *floats*: elements the layout engine is free to place wherever it fits best on the page, rather than pinning exactly where you wrote the code. That's a deliberate design choice (it avoids ugly page breaks splitting a figure across two pages), but it's also the single most common source of "why did my figure end up three pages later than I put it" confusion for anyone coming from a word processor, where images stay exactly where you drop them.

This chapter covers `\includegraphics` and the `figure` environment for the basic case, placement specifiers and `\clearpage` for controlling *where* floats actually land, vector-vs-raster format choice, and `subcaption` for multi-panel figures — the building blocks for the labeled, cross-referenceable figures Chapter 8 will start pointing back to by number.

Objective 7

Include images, caption and label them correctly, control where floats land on the page, and lay out multi-panel figures.

Including a Basic Figure with `\includegraphics`

Getting an image onto the page at all is the first hurdle — LaTeX has no native concept of an image file, so `\includegraphics` (from the `graphicx` package) is the command that bridges the gap, and wrapping it in a `figure` environment is what turns a bare image into a captioned, labeled, referenceable float.

Worked Example 7.1

```
\documentclass{article}

\usepackage{graphicx}

\begin{document}

    \begin{figure}[htbp]
        \centering
        \includegraphics[width=0.6\textwidth]{example-image}
        \caption{A sample figure.}
```

```
        \label{fig:sample}
    \end{figure}

\end{document}
```

Nuance 7.1

`example-image` is a built-in Overleaf/LaTeX placeholder (from the `mwe` package family) that renders without needing to upload a real file — useful while drafting before you have your actual figures ready.

Float Placement Specifiers

That `[htbp]` in the figure's opening brackets looks like a fixed instruction, but LaTeX treats it as a ranked list of acceptable spots rather than a command to obey exactly — which is exactly why a figure can end up somewhere you didn't ask for.

Important 7.1

The `[htbp]` argument is a *suggestion*, not a command:

- `h` (here),
- `t` (top of page),
- `b` (bottom),
- `p` (its own page).

LaTeX will override your preference if the figure doesn't fit — this is intentional, not a bug. Adding `!` (e.g. `[!htbp]`) tells LaTeX to relax some of its usual placement restrictions, which helps but doesn't guarantee "exactly here."

Controlling Where Floats Actually Land

Sometimes tweaking the placement letters isn't enough and a figure still queues up, drifting further from where you wrote it as body text accumulates — for that there are commands that force the issue outright instead of just suggesting. Placement specifiers only *suggest* where a float should land — sometimes LaTeX still queues one up and drops it several pages later, once enough body text has accumulated to make room.

Nuance 7.2

Three related commands, each doing a different amount of work:

- `\newpage`
  - Starts a new page. Doesn't touch any floats still waiting to be placed.
- `\clearpage`
  - Starts a new page *and* forces every pending float to flush and place before continuing. This is the actual fix for "my figure floated away from where I put it" — reach for it right before a section that shouldn't inherit the previous section's leftover floats.

- `\cleardoublepage`
  - The same as `\clearpage`, but also guarantees the next page is odd-numbered, inserting a blank page if needed. This only matters in two-sided (`twoside`) documents — see Ch. 20's `\frontmatter`/`\mainmatter`/`\backmatter`, which call this internally at every transition.

General 7.1 —

```
\section{Methods}
... several figures queued up, none placed yet ...

\clearpage
\section{Results}
% figures from the Methods section are forced to place here,
% before Results starts, instead of drifting further into the document
```

Cross-Check 7.1 — Cross-Check

See `supplementary_materials.md` for a full before/after example — a document with a genuinely stuck float, and the one-line fix.

Vector vs. Raster Images

Once a figure lands where you want it, the next question is what kind of image file to feed `\includegraphics` in the first place — the format you choose decides whether it still looks crisp at print size or starts looking blurry the moment it's scaled up.

Nuance 7.3

- **Vector** (PDF, EPS): scales to any size with no quality loss — ideal for line plots, diagrams, schematics.
- **Raster** (PNG, JPG): fixed resolution — fine for photos or screenshots, but can look blurry if scaled up.

For anything you generate yourself, export as PDF whenever the source tool allows it.

Handling Legacy `.eps` Figures

Vector is the right call in principle, but not every figure you inherit was exported in a format pdfLaTeX can actually read directly, and `.eps` files are the format most likely to turn up from older plotting tools.

Nuance 7.4

Older figures — especially ones exported from `MATLAB` or `gnuplot` sometimes only come as `.eps` files. It is a vector format `pdfLaTeX` can't include directly (only PDF/PNG/JPG).

If you're stuck with one and can't re-export it, `\usepackage{epstopdf}` converts it to PDF automatically at compile time:

General 7.2 —

```
\usepackage{epstopdf}
...
\includegraphics[width=0.6\textwidth]{old_plot.eps}
```

Important 7.2

This needs "shell escape" enabled for your compiler — if the conversion silently fails, that's usually why. The better fix, when possible, is re-exporting the figure as PDF from its source tool in the first place; treat `epstopdf` as a fallback for a figure you can't regenerate, not a habit.

Multi-Panel Figures with `subcaption`

A single `\includegraphics` call handles one image, but plenty of results are best shown as several related panels side by side under one shared caption — that's a layout plain `figure` can't do alone, and `subcaption` is the package built for it.

Worked Example 7.2

```
\documentclass{article}
\usepackage{subcaption}

\begin{document}
    \begin{figure}[htbp]
        \centering
        \begin{subfigure}{0.45\textwidth}
            \centering
            \includegraphics[width=\linewidth]{example-image-a}
            \caption{First panel}
            \label{fig:sub1}
        \end{subfigure}
        \hfill
        \begin{subfigure}{0.45\textwidth}
            \centering
            \includegraphics[width=\linewidth]{example-image-b}
            \caption{Second panel}
            \label{fig:sub2}
        \end{subfigure}
        \caption{Two panels side by side.}
        \label{fig:combined}
    \end{figure}
```

```
\end{document}
```

Nuance 7.5

Each `subfigure` gets its own caption/label (`fig:sub1`, `fig:sub2`); the outer `figure` gets an overall caption/label (`fig:combined`) for referring to the pair as a whole.

Nuance 7.6

The `caption` package (load it alongside `subcaption` above, which already depends on it) lets you adjust how every caption in the document looks — font size, and the separator between "Figure 1" and the caption text:

General 7.3 —

```
\usepackage{caption}
...
\captionsetup{font=small, labelsep=colon}
% "Figure 1: A sample figure." instead of the default
% "Figure 1  A sample figure."
```

Set this once in the preamble to apply it document-wide, or scope it locally by wrapping it and the figure together in an extra pair of `{ }` — the same scoping trick Ch. 19 uses for `\renewcommand{\arraystretch}`.

Important 7.3

The `subfigure` is deprecated. `subcaption`, which this chapter uses throughout, is its modern replacement; if you see `\usepackage{subfigure}` in an old template or forum answer, swap it for `subcaption` instead.

Exercise 7.1

1. Build a 2×2 subfigure grid: extend the example above to four subfigures, arranged two per row. To force the third and fourth subfigures onto a new row, insert `\par` (or `\\`) right after the second `\end{subfigure}`, before starting the third `\begin{subfigure}` — then repeat the two-subfigure pattern for the bottom row. Just pressing Enter/leaving a blank line is **not** enough: LaTeX treats a newline as an ordinary space, not a line break, so without `\par`/`\\` your third subfigure will just sit awkwardly next to the second.
2. Deliberately create a stuck float:
   - Add three or four figures in a row with no body text between them, followed by a new `\section`. Recompile and note where they actually land — then add `\clearpage` right before the new section and recompile again to see the difference.

Capstone Update 7

Insert a placeholder figure into your paper (using `example-image` is fine for now) with a proper caption and label — this is what Ch. 8 will cross-reference. If you've got more than one figure queued up by the time you assemble the full paper, use `\clearpage` to make sure they land where you intend rather than drifting into the next section.

Chapter 8 — Cross-Referencing, Labels & Hyperlinks

Every figure, table, and section you've labeled so far (`fig:sample`, `tab:sample`, and so on, from Chapters 6–7) has been sitting unused — a `\label` on its own does nothing until something else points back to it with `\ref`. Manually typing "see Figure 3" is exactly the kind of thing that goes stale the moment you insert a new figure earlier in the document and every number after it shifts by one; automatic cross-referencing exists specifically so that renumbering is the compiler's problem, not yours.

This chapter covers `\ref`/`\cref` for that automatic numbering (including why `cleveref`'s load order relative to `hyperref` matters), the labeling-prefix convention (`fig:`, `tab:`, `sec:`, ...) used consistently throughout the rest of the guide, and `\hypersetup` for turning `hyperref`'s default loud link-boxes into the quiet colored-text links expected in a submitted PDF.

Objective 8

Reference figures, tables, and sections by number automatically (so they never go stale), and set up hyperlinks that look professional instead of like a default web browser's blue-boxed mess.

Automatic Numbering with `\ref` and `\cref`

A `\label` is inert on its own. It only becomes useful once something points back to it by number, and that's what the reference commands are for. `cleveref`'s `\cref`/`\Cref` build on plain `\ref` by also supplying the right noun ("Figure", "Table", "Section") automatically, so renumbering — or retyping the wrong word — is never something you do by hand.

Worked Example 8.1

```
\documentclass{article}
\usepackage{hyperref}
\usepackage{cleveref}

\begin{document}
    As shown in \cref{fig:sample} and \cref{tab:sample}.
    The results support the hypothesis.
```

```
    See \Cref{tab:sample} at the start of a sentence (capitalized).
\end{document}
```

Important 8.1

- `\ref{fig:sample}` gives you just the number,
- `\cref{fig:sample}` (from `cleveref`) gives you the number *and* automatically prepends "Figure"/"Table"/"Section" as appropriate — one less thing to hand-type and get out of sync.
- `\Cref` capitalizes it for sentence starts.

Important 8.2

Load `hyperref` near the very end of your preamble, and load `cleveref` immediately *after* `hyperref`. Getting this backwards (loading `cleveref` before `hyperref`) is a classic source of broken or missing links — package load order matters more in LaTeX than in most other languages you may have used.

Styling Hyperlinks with `\hypersetup`

`hyperref` makes every cross-reference, citation, and URL clickable, but its out-of-the-box styling — thick colored boxes around every single link — reads like a rough draft rather than a submission-ready PDF. A single `\hypersetup` call swaps that for quiet, professional-looking links instead.

Nuance 8.1

By default, `hyperref` draws loud colored boxes around every link and citation — fine on screen, unprofessional in a printed or submitted PDF. Set this up properly instead:

Important 8.3

```
\hypersetup{
    colorlinks=true,
    linkcolor=blue,
    citecolor=blue,
    filecolor=blue,
    urlcolor=blue,
}
```

`colorlinks=true` colors the link *text* rather than boxing it. For a fully print-ready PDF with no visible link styling at all, use `hidelinks` instead.

General 8.1 — Forward Note

`\eqref` for equation cross-references is coming in Ch. 9 once equations are introduced — the same `\label`/`\ref` mechanics apply.

The Labeling-Prefix Convention

Naming every `\label` consistently can look like a cosmetic habit, but in a document with hundreds of labels spread across a dozen chapters, it's what keeps a reference like `\label{intro}` from meaning three different things and colliding with itself.

Nuance 8.2

Every `\label` in this guide so far has used a prefix — `fig:`, `tab:`, and so on — without ever naming the convention directly. Worth making explicit now that cross-referencing is the whole point of this chapter:

Important 8.4

Prefix	For
`fig:`	Figures
`tab:`	Tables
`eq:`	Equations (Ch. 9)
`sec:/ch:`	Sections/chapters
`thm:`	Theorems (Ch. 11)
`alg:`	Algorithms (Ch. 12)
`def:`	Definitions (Ch. 11)

General 8.2 — What the prefix actually does/doesn't do

`cleveref` does *not* read the prefix text to decide whether to print "Figure" or "Table." It determines that from the *counter* that was stepped when `\label` was called. A `\caption` inside a `figure` steps the `figure` counter, a `\section` steps the `section` counter, and so on, via `\refstepcounter` working behind the scenes.

Important 8.5

The prefix is a discipline **for you, the author**. It prevents name collisions (`\label{intro}` could be a section, a figure, or a theorem in a 23-chapter document) and makes every `\ref`/`\cref` call in your source instantly scannable, without needing to jump to the label to know what it points to.

Exercise 8.1

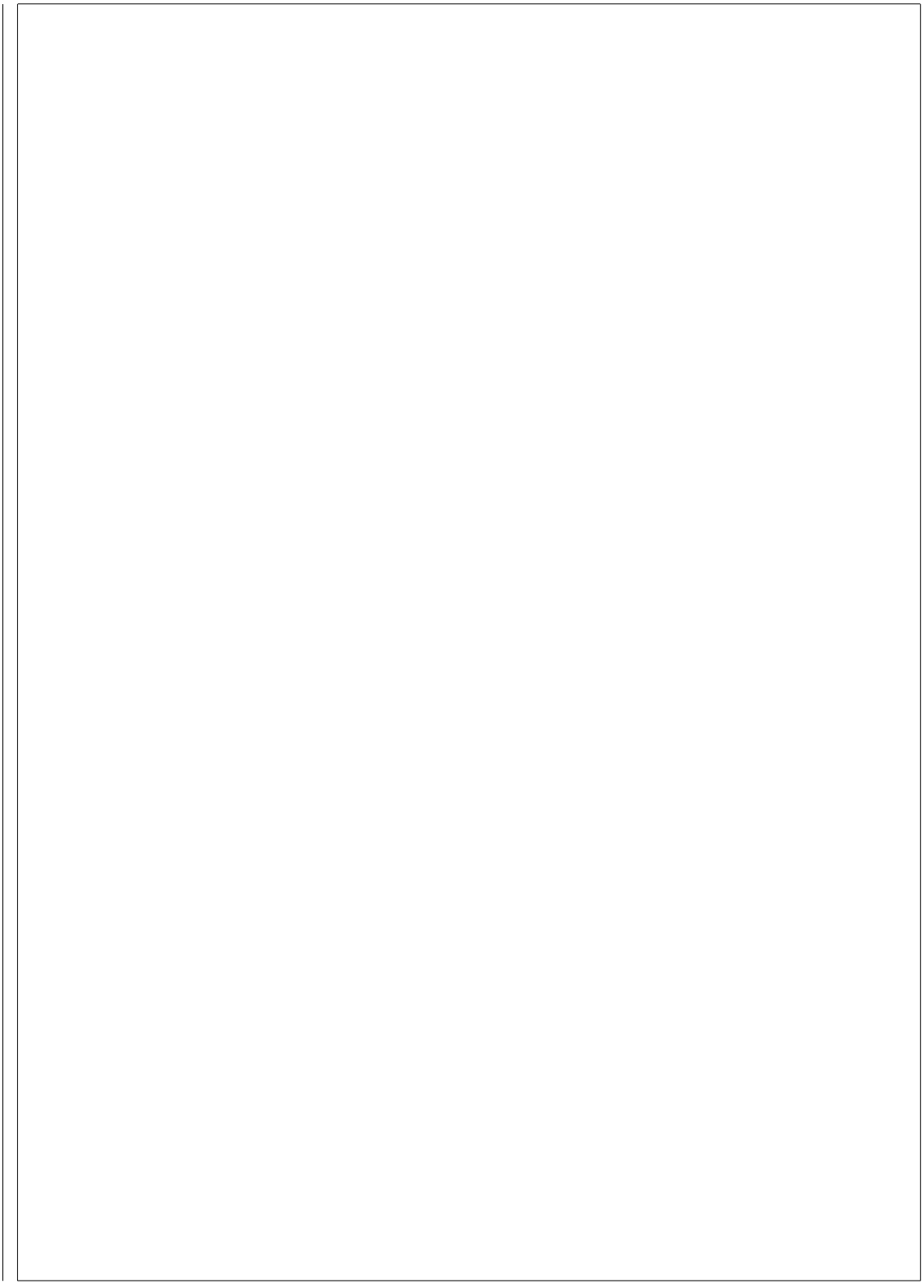
1. Take a document with a labeled figure and table where one `\label` and its matching `\ref`/`\cref` have been deliberately mistyped (e.g. `\label{fig:sampl}` vs `\cref{fig:sample}`). Recompile, note the `??` that appears in place of the number, and fix the mismatch.
2. Add the `\hypersetup` block above and recompile. Compare the look of your links before and after.

3. Bonus: audit the labels in your capstone paper so far. Confirm every one uses a consistent prefix (`fig:`, `tab:`, `sec:`, etc.), and fix any that don't.

Capstone Update 8

Cross-reference the result table (Ch. 6) and figure (Ch. 7) from your paper's body text using `\cref`, and apply the `colorlinks` setup so your capstone PDF looks presentation-ready rather than default-browser-blue.

Part 3: Math Mechanics (Chapters 9–10)



Chapter 9 — Inline & display math basics/fonts & units

Math is where LaTeX earns its reputation — an equation that would be a fight with an equation editor elsewhere is a few lines of markup here, and it renders consistently every time instead of drifting out of alignment across a long document.

This chapter covers the basics everything else in Part 3 builds on: inline vs. display math, Greek letters and sub/superscripts, and the font/unit conventions (roman vs. italic, `siunitx`, `bm`) that separate a professionally typeset paper from an obviously beginner one.

Objective 9

Typeset inline and display equations correctly, use Greek letters and sub/superscripts, and get the font and unit conventions right from the start — the habits that separate a professionally typeset paper from an obviously beginner one.

Inline vs. display math

The same equation can sit inline within a sentence, or be set apart on its own numbered line — LaTeX uses different syntax for each, and mixing them up is an easy first mistake.

Worked Example 9.1

```
\documentclass{article}
\usepackage{amsmath}

\begin{document}
    Einstein's mass-energy equivalence, written inline, is $E = mc^2$.
    The same relationship, set apart and automatically numbered:

    \begin{equation}
        E = mc^2
        \label{eq:mass-energy}
    \end{equation}
```

```
    Two related equations, aligned at the equals sign and each numbered:

    \begin{align}
        F &= ma        \label{eq:newton2} \\
        W &= Fd        \label{eq:work}
    \end{align}
\end{document}
```

Important 9.1

- `$...$` is inline math — it flows within a sentence.
- `equation` is display math with automatic numbering. It can be references with `\eqref{eq:mass-energy}` or `\cref{eq:mass-energy}`. The same `\label`/reference mechanics from Ch. 8, just applied to an equation.
- `\eqref` wraps the number in parentheses (`(1)`), which is the standard convention for equation references; `\cref` instead produces "Equation (1)" with the word spelled out. Either is correct, just pick one convention and stay consistent.
- Need an equation with *no* number at all? `equation*` is the starred, unnumbered variant. See Ch. 5's "star convention" nuance if that's new to you.
- `align` (from `amsmath`, loaded above) numbers *every* line and aligns them at the `&` marker. The & signa is typically placed right before the `=` sign.

Greek letters and sub/superscripts

Variable names and exponents in physics and math notation lean heavily on Greek letters and sub/superscripts, both typed with plain-text shortcuts rather than any special input method.

Worked Example 9.2

```
\documentclass{article}

\begin{document}
    The decay follows $N(t) = N_0 e^{-\lambda t}$, where $\lambda$ is the
    decay constant.
    Summing over $i = 1, \dots, n$:
    \[
        S = \sum_{i=1}^{n} x_i^2
    \]
\end{document}
```

Important 9.2

- Greek letters are typed by name:
  - `\alpha`, `\beta`, `\gamma`, `\theta`, `\pi`, `\sigma`, `\omega` for lowercase,
  - `\Gamma`, `\Delta`, `\Theta`, `\Pi`, `\Sigma`, `\Omega` (capitalized command) for the uppercase forms that differ from Latin letters.

- Subscripts use `_`, superscripts use `^`; wrap multi-character sub/superscripts in braces (`x_{i,j}^{2}`), since `x_i,j^2` would only attach `i` to the subscript.

Italics vs. Roman in Math

Math mode italicizes every letter by default, which is correct for a variable but wrong for a handful of other things that just happen to also be letters.

Nuance 9.1

Letters in math mode render in italic by default, which is correct for single-letter variables (x, y, α), but wrong for a few other things:

- **Vectors/matrices** should be bold, not italic; see the `bm` nuance below.
- **Units** should be upright/roman, not italic; see `siunitx` below.
- **Plain English words** inside math mode (like "if" in a `cases` block, covered in Ch. 10) need `\text{}`, or LaTeX renders them as multiplied variables.

Worked Example 9.3

```
\documentclass{article}

\begin{document}
    % correct: x is italic automatically, no extra command needed
    $x^2 + y^2 = r^2$

    % correct: \mathrm{} for upright text/units inside an equation
    $F = 5\ \mathrm{N}$

    % WRONG: don't reach for \textit inside math mode
    $\textit{x}^2$
\end{document}
```

Important 9.3

Never use `\textit{}` inside math mode to italicize a variable. `\textit{}` is a *text-mode* command; using it in math mode produces inconsistent spacing and kerning compared to LaTeX's own math italics. Just write `$x$` — it's already italic automatically.

Formatting Units

A number with a unit can be typed by hand correctly, but staying consistent about it across a 40-page document with hundreds of units is the part that actually goes wrong.

Nuance 9.2

You *can* write units manually, and it's not wrong:

Worked Example 9.4

```
\documentclass{article}
\usepackage{amsmath}

\begin{document}
    % manual approach - technically correct
    $10\,\text{keV}$
    % or, equivalently
    $10\,\mathrm{keV}$
\end{document}
```

Both use tools you already have: `\,` for the thin space between number and unit, `\text{}`/`\mathrm{}` to keep the unit upright instead of italic. Nothing here is broken.

`siunitx`: The solution

The catch is consistency, not correctness. Across a 40-page thesis with hundreds of units, it's easy to forget the `\,` on line 200 when you remembered it on line 20, mix `\text{}` and `\mathrm{}` inconsistently, or format `\times 10^{-3}` one way in one equation and another way elsewhere. None of these individually breaks compilation, they just make the document look inconsistent, which readers (and reviewers) notice even if they can't say why.

Nuance 9.3

The `siunitx` package's `\SI{value}{unit}` command removes that risk entirely by enforcing the same spacing and formatting every time, automatically:

Worked Example 9.5

```
\documentclass{article}
\usepackage{siunitx}

\begin{document}
    The measured velocity was \SI{5}{\meter\per\second}.
    The sample mass was \SI{10.2}{\kilogram}.
    A unit can also be shown on its own, without a value, using
    \si{\meter\per\second}.
\end{document}
```

`\SI{}{}` inserts the correct thin space between number and unit and renders the unit upright. Manual `\,\text{}` is fine for a one-off; `siunitx` is what you want the moment a document has more than a handful of units to stay consistent across.

Bold Vectors & Greek Letters with bm

LaTeX's standard bold command quietly fails on Greek letters, which is a surprising first encounter for anyone typesetting a bold coefficient vector because the default LaTeX font doesn't cover Greek symbols.

Nuance 9.4

For bold vectors or bold Greek symbols (common in statistics/ML notation — a coefficient vector $\boldsymbol{\beta}$, for instance), use the bm package:

Worked Example 9.6

```
\documentclass{article}
\usepackage{bm}

\begin{document}
    The regression coefficients $\bm{\beta}$ minimize the loss function.
    A bold Latin vector works the same way: $\bm{v}$.
\end{document}
```

\boldsymbol{} (from amsmath) is a lighter alternative that covers most Greek letters too, but bm handles more symbol combinations, so it's the one used throughout this guide.

Putting It Together

A single short derivation that combines display equations, \bm{}, and \SI{}{} in one place is worth more than seeing each of them in isolation.

Worked Example 9.7

```
\documentclass{article}
\usepackage{amsmath}
\usepackage{siunitx}
\usepackage{bm}

\begin{document}
    Newton's second law relates force and acceleration:
    \begin{equation}
        \mathbf{F} = m\bm{a}
        \label{eq:newton2}
    \end{equation}
    where $m$ is mass in \si{\kilogram} and $\bm{a}$ is acceleration in
    \si{\meter\per\second\squared}.

    A measured force of \SI{12.5}{\newton} acting on a \SI{2}{\kilogram}
    mass gives an acceleration of \SI{6.25}{\meter\per\second\squared}.
\end{document}
```

Exercise 9.1

1. Transcribe these into LaTeX (use `align` for anything spanning more than one line):
   - $a^2 + b^2 = c^2$
   - $\int_0^\infty e^{-x}dx = 1$ *(we'll fix this integral's spacing properly in Ch. 10)*
   - $\lim_{x\to 0} \frac{\sin x}{x} = 1$
   - A vector dot product: $\boldsymbol{u} \cdot \boldsymbol{v} = |\boldsymbol{u}||\boldsymbol{v}| \cos\theta$
   - A bold Greek coefficient: $\hat{y} = \boldsymbol{\beta}^T \boldsymbol{x}$
2. Find a professionally typeset equation from a textbook or paper in your field (a screenshot is fine), and write the LaTeX that would reproduce it. Check your version against the rules above: is anything that should be bold left plain? Any unit set in italics that shouldn't be?

Cross-Check 9.1 — Cross-Check

Sample target equations are collected in `supplementary_materials.md` — see "Ch. 9 — Reverse-Engineering Sample Targets."

Capstone Update 9

Typeset the single most important equation in your capstone paper — the one that defines your key result or model — using correct roman/bold conventions and `\SI{}{}` for any units involved.

Chapter 10 — Advanced Math: Matrices, Multi-line Equations, Cases

Chapter 9 covered single equations; real derivations are rarely just one line — they're matrices, branching definitions, and multi-step algebra that each need their own environment to typeset cleanly.

This chapter covers `pmatrix`/`bmatrix`/`vmatrix` for matrices, `cases` for piecewise functions, `align`/`split` for multi-step derivations, and the delimiter-sizing and spacing details that make a finished derivation look deliberately typeset rather than dumped from a text editor.

Objective 10

Typeset matrices, piecewise functions, and multi-step derivations, and control delimiter sizing and spacing so equations look deliberate rather than cramped or oversized.

Matrices

A matrix is just a grid inside brackets, and the bracket style is a one-word swap once you know the environment name to change.

Worked Example 10.1

```
\documentclass{article}
\usepackage{amsmath}

\begin{document}
    \[
        A = \begin{pmatrix}
                1 & 2 \\
                3 & 4
        \end{pmatrix}
    \]
\end{document}
```

Nuance 10.1

`pmatrix` gives parentheses; swap the environment name for a different bracket style

- `bmatrix` for square brackets,
- `vmatrix` for a determinant (single vertical bars),
- `Vmatrix` for double vertical bars.

The `&` (column separator)/`\\` (row separator) syntax stays the same across all of them.

Piecewise Functions with `cases`

A function defined differently across a few conditions needs its branches aligned under one shared brace, which is exactly what the `cases` environment is for.

General 10.1 — Usecase of ‘cases‘

```
\[
    f(x) =
    \begin{cases}
        x^2  & \text{if } x \geq 0 \\
        -x^2 & \text{if } x < 0
    \end{cases}
\]
```

Nuance 10.2

Note the `\text{}` around “if”. Without it, “if” renders in italic math font as though it were two multiplied variables (i times f) and it’ll look wrong and read worse. Any plain-English word inside math mode needs `\text{}` around it.

Multi-Step Derivations

A derivation that unfolds over several lines needs every line aligned at the same operator, and a choice about whether each intermediate step gets its own equation number.

Worked Example 10.2

```
\documentclass{article}

\begin{document}
    \begin{align}
    (a+b)^2 &= (a+b)(a+b)            \\
            &= a^2 + ab + ba + b^2   \\
            &= a^2 + 2ab + b^2
    \end{align}
\end{document}
```

Important 10.1

`align` numbers every line by default. If you only want the *final* line numbered, wrap a `split` block inside a single `equation` instead:

Worked Example 10.3

```
\begin{equation}
    \begin{split}
    (a+b)^2 &= (a+b)(a+b)      \\
        &= a^2 + ab + ba + b^2 \\
        &= a^2 + 2ab + b^2
    \end{split}
\end{equation}
```

Use `align*` (with an asterisk) if you want *no* line numbered at all — the same starred-variant pattern as `\section*{}` from Ch. 5, applied here to equations.

Controlling Fraction Size

A fraction's default size depends on whether it's inline or in a display equation, and three commands exist for overriding that default in either direction.

Nuance 10.3

`\tfrac`, `\dfrac`, and `\displaystyle`

A fraction's size normally depends on context: `$\frac{1}{2}$` written inline renders small and compact to fit the line; the same `\frac{1}{2}` inside a display equation (`\[...\]`, `equation`, `align`) renders larger, in what's called "display style." Three commands (all from `amsmath`) let you override that default in either direction:

- **`\displaystyle`** forces display-style sizing even inline — `$\displaystyle\frac{1}{2}$` renders full-size in the middle of a sentence. Useful occasionally, but can disrupt line spacing if overused.
- **`\dfrac{}{}`** is shorthand for exactly that, applied just to one fraction: `\dfrac{1}{2}` always renders at display size, wherever it's used.
- **`\tfrac{}{}`** is the opposite shorthand: always renders at the smaller, inline ("text style") size, even inside a display equation — handy for a small fraction that would otherwise look oversized as part of a larger expression.

Delimiter Sizing

`\left`/`\right` auto-sizing is convenient but not always the right call — a simple fraction wrapped in it often ends up more oversized than the expression actually needs.

Nuance 10.4

Size Matters: delimiter sizing
`\left(` and `\right)` auto-size to match their contents — convenient, but easy to overdo. Wrapping a simple fraction in `\left( \right)` often produces oversized, over-spaced parentheses:

```
% often too large / spaced-out for something this simple
$\left( \frac{1}{2} \right)$

% better: a manual, smaller, tighter size
$\bigl( \tfrac{1}{2} \bigr)$
```

The manual sizing commands, from smallest to largest, are `\bigl`/`\bigr`, `\Bigl`/`\Bigr`, `\biggl`/`\biggr`, `\Biggl`/`\Biggr` (each pair is the left/right delimiter respectively). Pick whichever size actually matches your content instead of defaulting to `\left`/`\right` every time. Note the deliberate use of `\tfrac` rather than `\frac` here too — per the nuance above, it keeps the fraction at its natural compact size instead of forcing display-style sizing into an already-tight expression.

One-Sided Delimiters

Showing a function evaluated between two bounds needs a tall bar on the right and deliberately nothing on the left to match it.

Nuance 10.5

one-sided delimiters: `\left. ... \right|`
Evaluating a function or antiderivative between two bounds needs a vertical bar sized to match the expression's height, but nothing on the left side to match it:

```
\[
    \int_0^1 x \, dx = \left. \frac{x^2}{2} \right|_{0}^{1} = \frac{1}{2}
\]
```

`\left.` (a left delimiter followed immediately by a period) tells LaTeX "size the right delimiter to match, but draw nothing on the left." `\right|` then draws a properly sized vertical bar, with the bounds as its subscript/superscript (`_{0}^{1}`). This is the standard way to show a function evaluated at bounds — you'll use it constantly whenever a derivation is followed by "evaluated from a to b."

Math Spacing

LaTeX doesn't insert any breathing room around math operators or differentials on its own — everything you see spaced out in a polished equation was spaced out on purpose.

Nuance 10.6

Math spacing: `\,` and `\!`
Differentials in integrals look cramped without a small gap before them:

```
% cramped - the "dx" runs right into "x"
$\int_0^1 x dx$

% correct - a thin space before the differential
$\int_0^1 x \, dx$
```

`\,` inserts a thin space; `\!` inserts a *negative* thin space (pulls two elements closer together) — useful for tightening nested exponents or an overly loose double integral. These are small, cosmetic details, but exactly the kind that make a derivation look deliberately typeset rather than dumped from a text editor.

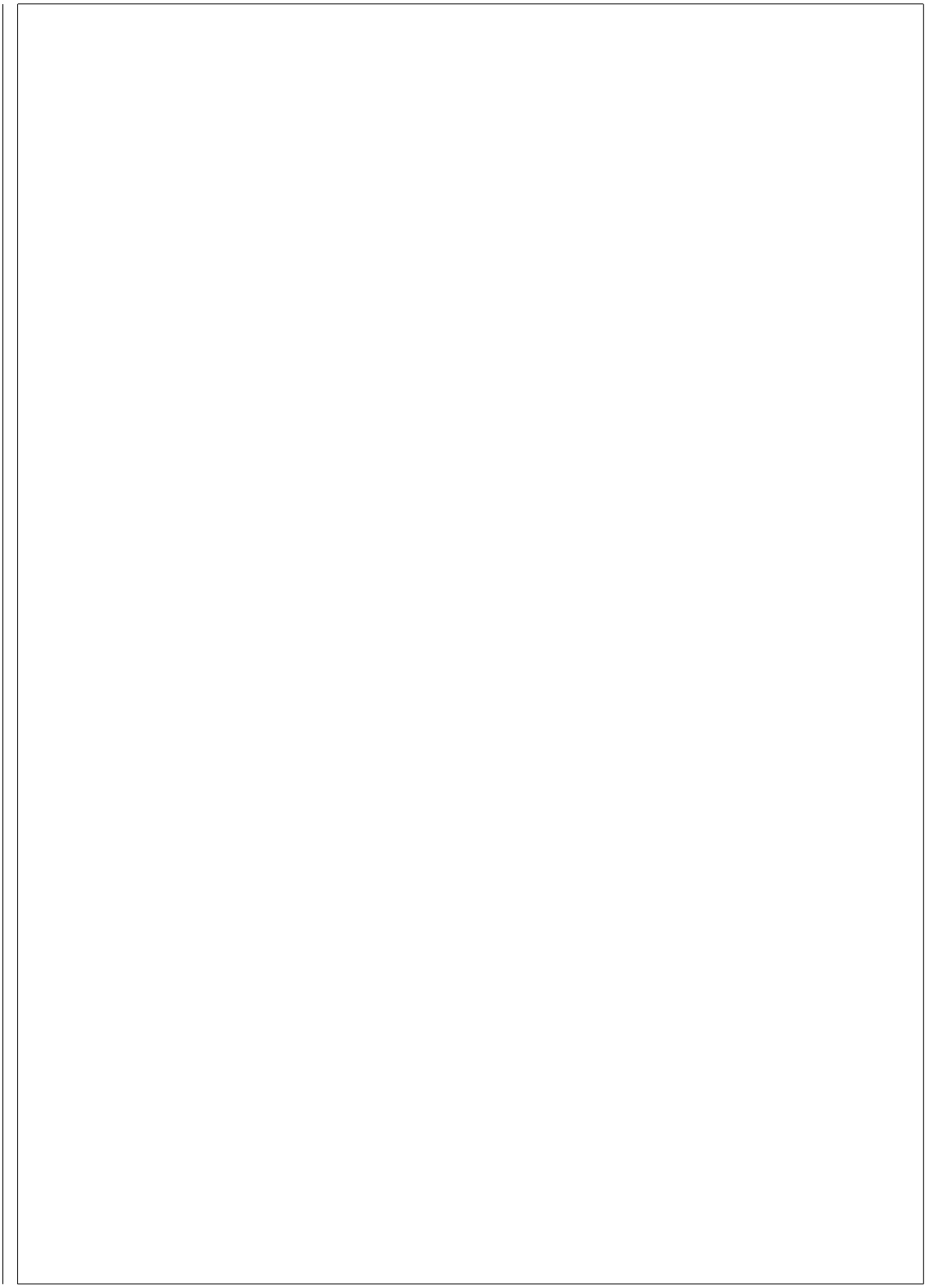
Exercise 10.1

1. Typeset the coefficient matrix and right-hand-side vector of a 2-equation linear system as a single matrix equation, using `pmatrix`.
2. Typeset a 3-branch piecewise function with `cases`, remembering `\text{}` around any English words.
3. Take one `\left( \right)` expression (your own, or from Ch. 9's exercises) and rewrite it using `\bigl/\bigr` instead — compare the rendered size.
4. Revisit the integral from Ch. 9's exercise 1 ($\int_0^\infty e^{-x}dx = 1$) and add the thin space: $\int_0^\infty e^{-x}\,dx = 1$.
5. Typeset $\int_0^2 x^2\,dx = \left.\frac{x^3}{3}\right|_0^2 = \frac{8}{3}$ using `\left.`/`\right|`.

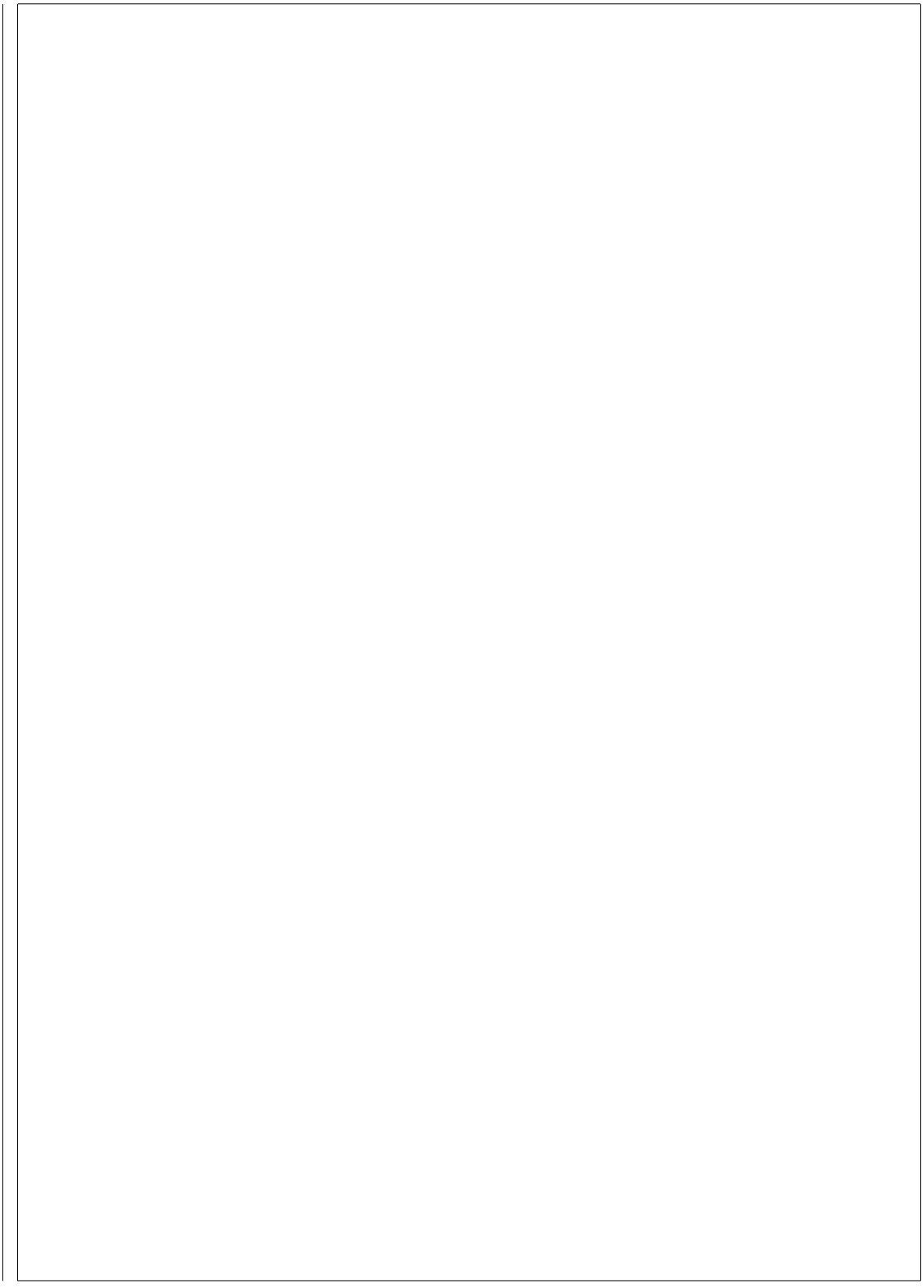
Capstone Update 10

Add a real derivation relevant to your capstone topic (2–4 steps is plenty) using `align` or `split`, and tighten the spacing around any integrals or products with `\,`.

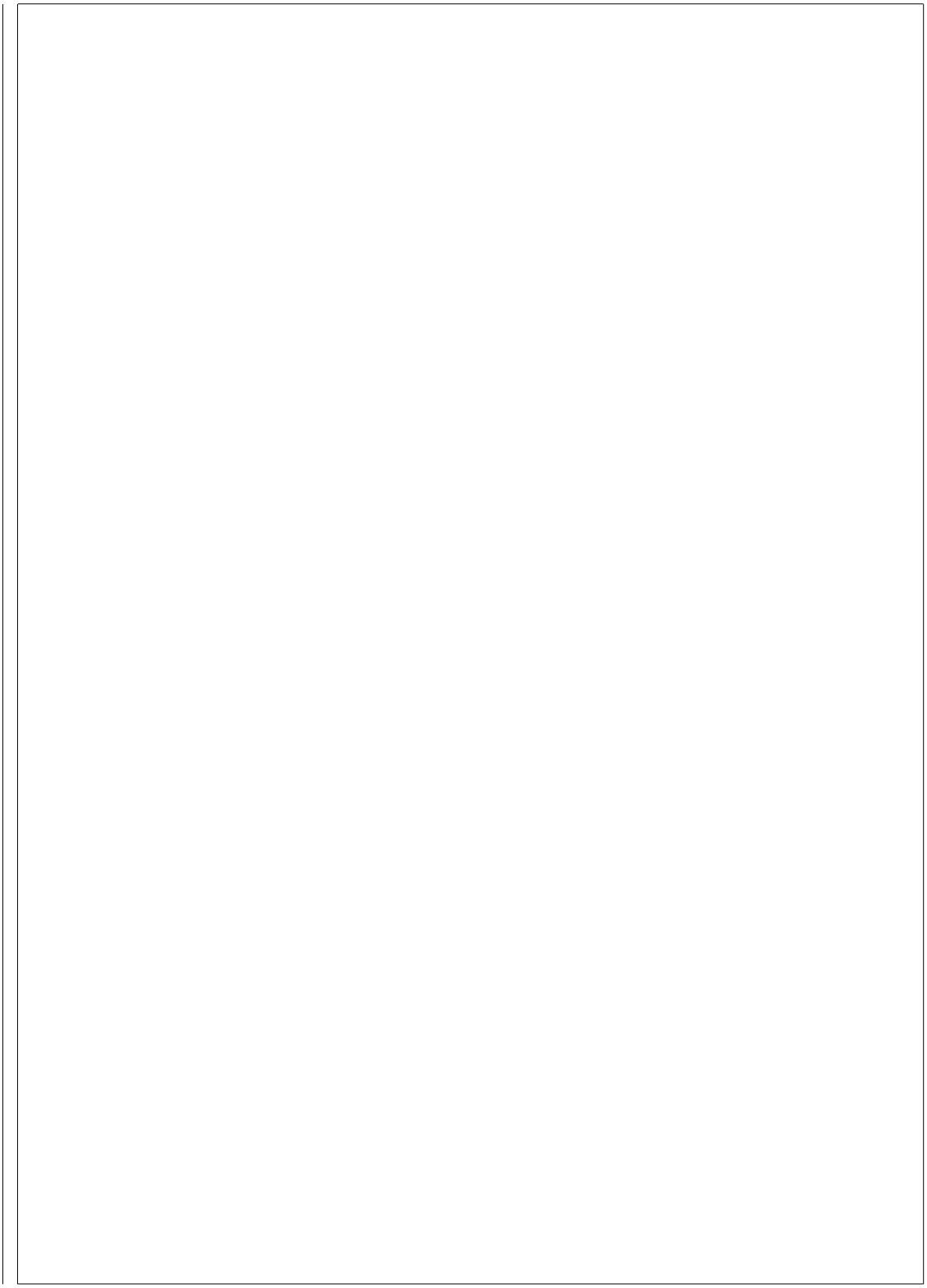
End of Part 3 (Ch. 9–10). Next: Part 4 — Math Structures & Algorithms (Ch. 11–12).



Appendix



Reference material, not narrative teaching — everything below assumes you've already read the chapter it points back to, and exists purely so you don't have to go hunting for a command you've seen once before. Where something is fully explained elsewhere in the guide, this appendix links to it rather than re-teaching it; a few items (page geometry, dimension keywords, spacing glue) are genuinely new, since they didn't fit naturally into any single chapter's narrative.



A.1 — Symbol & Command Cheat Sheet

Greek letters — type the name; capitalize the command for the uppercase form where it differs from a Latin letter:

Lowercase	Command	Uppercase	Command
α	`\alpha`	—	*(looks like Latin A)*
β	`\beta`	—	*(looks like Latin B)*
γ	`\gamma`	Γ	`\Gamma`
δ	`\delta`	Δ	`\Delta`
ϵ / ε	`\epsilon` / `\varepsilon`	—	*(looks like Latin E)*
θ	`\theta`	Θ	`\Theta`
λ	`\lambda`	Λ	`\Lambda`
μ	`\mu`	—	*(looks like Latin M)*
π	`\pi`	Π	`\Pi`
σ	`\sigma`	Σ	`\Sigma`
ϕ / φ	`\phi` / `\varphi`	Φ	`\Phi`
ω	`\omega`	Ω	`\Omega`

Common math operators — built in: `\sin`, `\cos`, `\tan`, `\log`, `\ln`, `\exp`, `\max`, `\min`, `\lim`, `\sup`, `\inf`. Need one that isn't built in (`argmin`, `rank`, `trace`, ...)? Declare it once with `\DeclareMathOperator` — full explanation and worked examples in Ch. 19.

Accents: `\hat{x}`, `\bar{x}`, `\dot{x}`, `\ddot{x}`, `\tilde{x}`, `\vec{x}`.

Bold and vectors: `\mathbf{}` bolds Latin letters only — it does *not* bold Greek letters (`\mathbf{\beta}` silently fails). Use `\bm{}` (the `bm` package) for bold Latin *and* Greek alike — full explanation in Ch. 9.

Delimiter sizing, smallest to largest: `\bigl(/\bigr)`, `\Bigl(/\Bigr)`, `\biggl(/\biggr)`, `\Biggl(/\Biggr)` — pick the size that matches your content instead of defaulting to auto-sizing `\left(/\right)` every time. Full explanation and the one-sided-delimiter variant (`\left. ... \right|`) in Ch. 10.

Spacing in math mode: `\,` (thin space, e.g. before a differential: `\int x \, dx`), `\!` (negative thin space, pulls things together) — Ch. 10. `\quad`/`\qquad` (fixed 1em/2em spaces, useful for separating side conditions like " $x > 0$") — see A.3 below.

A.2 — Table Formatting Cheat Sheet

Rules — old style vs. the field standard:

```
% dated - hand-drawn rules on every row, vertical bars
\begin{tabular}{|c|c|}
\hline
A & B \\
\hline
\end{tabular}

% field standard - booktabs, no vertical rules
\begin{tabular}{cc}
\toprule
A & B \\
\bottomrule
\end{tabular}
```

Full explanation in Ch. 6.

Merging cells: `\multicolumn{cols}{align}{text}` (built into LaTeX) spans columns; `\multirow{rows}{width}{text}` (needs `\usepackage{multirow}`) spans rows.

```
\begin{tabular}{lcc}
\toprule
\multirow{2}{*}{Sample} & \multicolumn{2}{c}{Measurement} \\
                        & Trial 1 & Trial 2 \\
\midrule
A & 10 & 12 \\
\bottomrule
\end{tabular}
```

Full explanation in Ch. 6.

Aligned decimals: `siunitx`'s `S` column type aligns every row's decimal point. Remember to wrap non-numeric header text in `{}`.

```
\begin{tabular}{l S[table-format=3.2]}
\toprule
{Sample} & {Value} \\
```

```
\midrule
A & 123.45 \\
B & 98.60  \\
\bottomrule
\end{tabular}
```

Full explanation in Ch. 6 and Ch. 9.

Wide or oversized tables: `sidewaystable` (from `rotating`) rotates a single float; the `landscape` environment (from `pdflscape`) rotates the whole physical page when even a rotated float isn't enough. Full explanation and examples in Ch. 6.

Scaling a table to fit the page: `\resizebox{\textwidth}{!}{...}` — use sparingly, since it shrinks the font along with everything else.

```
\resizebox{\textwidth}{!}{%
\begin{tabular}{cccccccc}
\toprule
Col1 & Col2 & Col3 & Col4 & Col5 & Col6 & Col7 & Col8 \\
\midrule
1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\
\bottomrule
\end{tabular}%
}
```

Multi-row header cell hack, paired with `\arraystretch`: nesting a small `tabular` inside a single cell wraps a long header onto two lines — reach for `multirow` first when it's a genuine data cell, and treat this as the pragmatic fallback for header labels specifically. Row height often needs loosening to match, via `\arraystretch` (full explanation in Ch. 19), scoped to just this table with an extra pair of `{ }`:

```
{\renewcommand{\arraystretch}{1.3}
\begin{tabular}{l @{} c}
\toprule
Sample & \begin{tabular}[c]{@{}c@{}}No. of\\detectors\end{tabular} \\
\midrule
A01 & 3 \\
\bottomrule
\end{tabular}
}
```

Other edge-case tools: `@{}` removes inter-column padding (used above, between `l` and the wrapped-header column); `tablefootnote` makes footnotes actually render inside a `table` float; `threeparttable` is a more formal alternative to `tablefootnote` if a table needs several notes at once. Full explanation in Ch. 6.

`minipage` — and how it differs from `subfigure`

Example 1 — two small tables side by side:

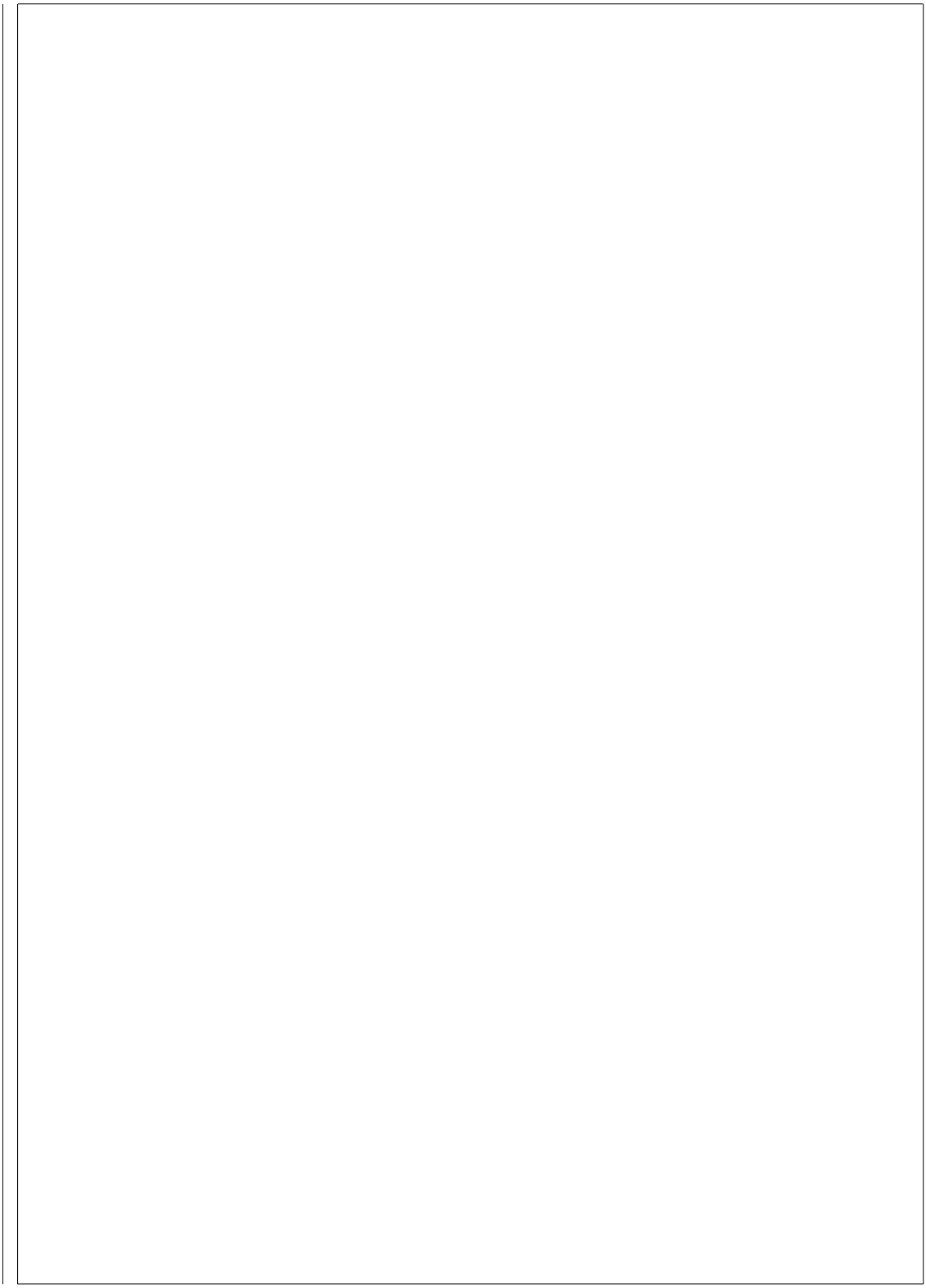
```
\begin{minipage}{0.48\textwidth}
\centering
```

```
\begin{tabular}{lc}
\toprule
Sample & Value \\
\midrule
A & 12.5 \\
\bottomrule
\end{tabular}
\end{minipage}
\hfill
\begin{minipage}{0.48\textwidth}
\centering
\begin{tabular}{lc}
\toprule
Sample & Value \\
\midrule
B & 7.3 \\
\bottomrule
\end{tabular}
\end{minipage}
```

Example 2 — a table next to explanatory text:

```
\begin{minipage}{0.55\textwidth}
\begin{tabular}{lc}
\toprule
Sample & Value \\
\midrule
A & 12.5 \\
\bottomrule
\end{tabular}
\end{minipage}
\hfill
\begin{minipage}{0.4\textwidth}
\small
A single measurement, included here as an example
of a table sitting beside its own explanatory note.
\end{minipage}
```

`minipage` vs. `subfigure`: both place things side by side, but they solve different problems. `subfigure` (Ch. 7, via `subcaption`) is specifically for multiple *numbered, captioned* image panels within one `figure` — each gets its own label (`fig:sub1`, `fig:sub2`), and the pair shares an overall caption. `minipage` is a general-purpose box with no captioning system of its own — reach for it for tables, mixed table-and-text layouts, or anything side-by-side that doesn't need individual sub-numbering. If you're laying out multiple images that should each get their own "Figure 1a"/"Figure 1b" style label, use `subfigure`; for everything else side-by-side, `minipage` is the simpler tool.



A.3 — Page Layout & Spacing Reference

(New material — this is genuinely reference-only content that didn't fit any single chapter's narrative, unlike the sections above.)

Page geometry — `geometry` overrides your document class's default margins:

```
\usepackage[margin=1in]{geometry}
% or, for individual sides:
\usepackage[left=1.5in, right=1in, top=1in, bottom=1in]{geometry}
% paper size, if you need something other than the class default:
\usepackage[a4paper, margin=1in]{geometry}
```

Set it once near the top of the preamble — it overrides most documentclass-level defaults. Useful the moment a university thesis format specifies exact margins (a common requirement this guide's authors ran into directly).

Dimension keywords — which one to reach for:

Keyword	Meaning	Typical use
`\textwidth`	Full width of the body text on the page	Sizing a figure/table to the whole page: `\includegraphics[width=\textwidth]{...}`
`\linewidth`	Width of the current line — adapts to context	Inside a `minipage`, `figure`, or list, where the available width is narrower than the full page
`\columnwidth`	Width of a single column	Specifically in two-column layouts (Ch. 20)

`\linewidth` is the more flexible default inside floats and lists, since it automatically adjusts; `\textwidth` stays fixed regardless of context, which occasionally causes an image to overflow a `minipage` if used there by mistake.

Spacing commands (outside math mode):

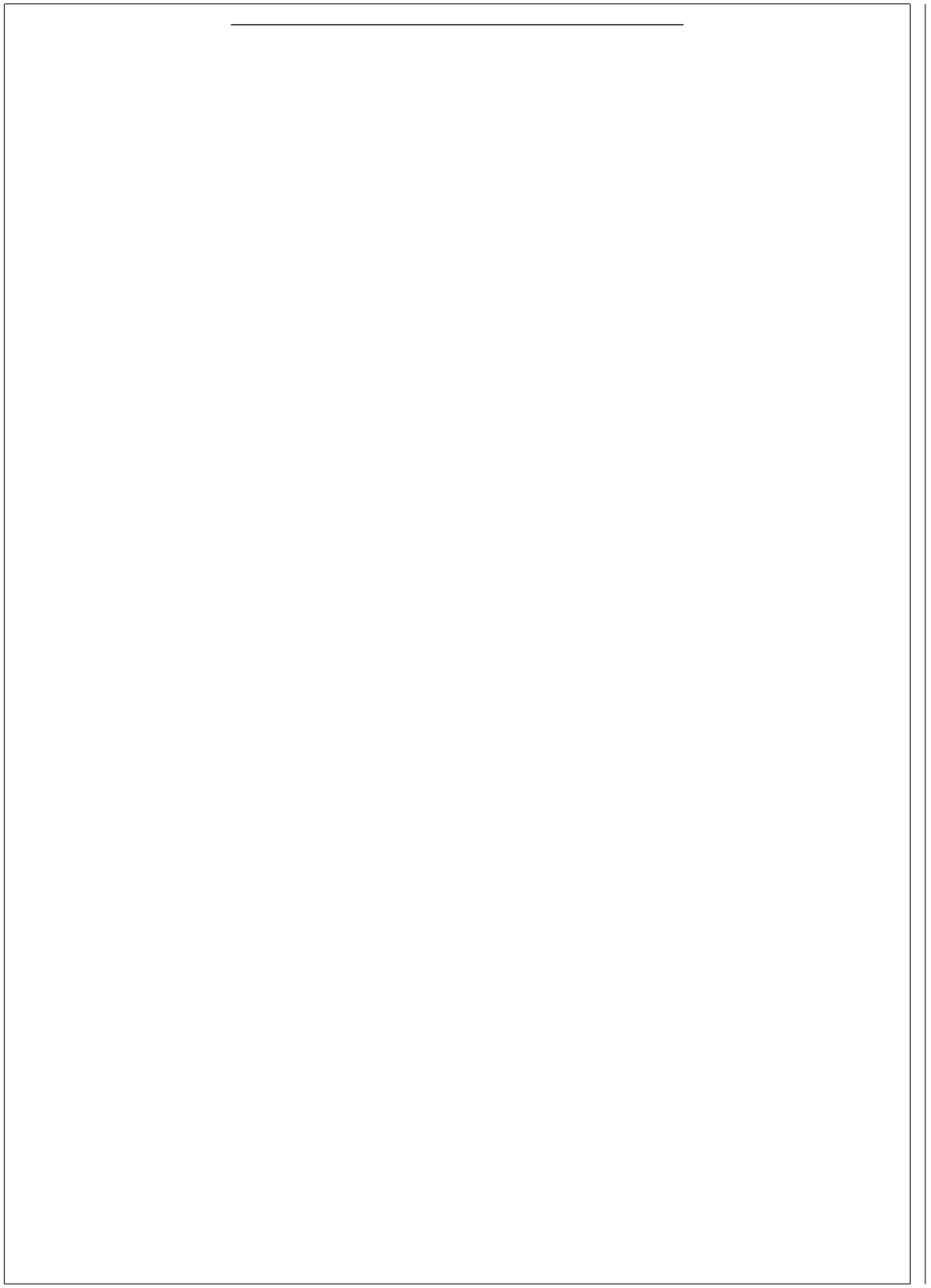
Command	Effect
`\hspace{1cm}`	Fixed horizontal space
`\hfill`	Flexible horizontal space — pushes content to opposite edges
`\vspace{1cm}`	Fixed vertical space
`\vspace*{1cm}`	Same, but survives at the top of a page (plain `\vspace` can get silently dropped there)
`\vfill`	Flexible vertical space — pushes content to the top/bottom of the page
`\hrule`	A horizontal line (rarely needed directly — use `booktabs` for tables, Ch. 6)
`\quad`	Fixed 1em space — common in math mode (Ch. 9–10), occasionally used for manual text indentation
`\qquad`	Fixed 2em space — same use cases as `\quad`, just wider

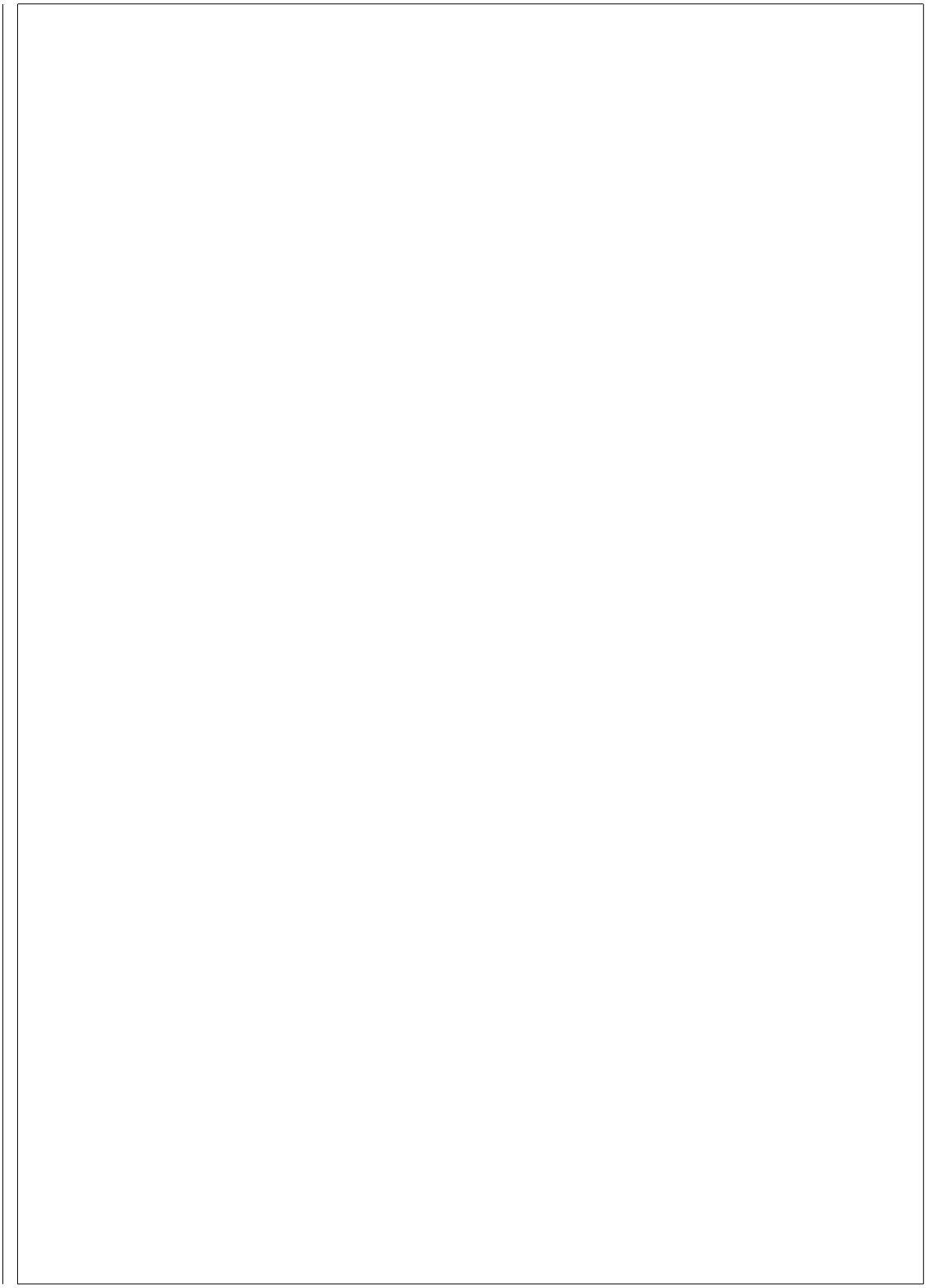
A.4 — Common Package Reference Table

Package	Purpose	First used	Load-order notes
`inputenc`	Declares UTF-8 source encoding (pdfLaTeX only)	Ch. 3	None
`amsmath`	Core math environments (`align`, `cases`, `split`), `\dfrac`/`\tfrac`	Ch. 9	Load before `amssymb`
`amssymb`	Extra math symbols, `\mathbb{}`	Ch. 19	—
`bm`	Bold Greek and Latin math symbols	Ch. 9	—
`siunitx`	`\SI{}{}`, aligned `S` table columns	Ch. 6, Ch. 9	—
`graphicx`	`\includegraphics`, image inclusion	Ch. 3 (exercise), Ch. 7 (taught)	—
`setspace`	`\doublespacing`/`\onehalfspacing`/`\singlespace`	Ch. 5	—
`enumitem`	Custom list labels and spacing	Ch. 6	—
`booktabs`	`\toprule`/`\midrule`/`\bottomrule`	Ch. 6	—
`multirow`	`\multirow{}{}{}`	Ch. 6	—
`rotating`	`sidewaystable`	Ch. 6	—
`pdflscape`	`landscape` environment (pdfLaTeX-safe)	Ch. 6	—
`tablefootnote`	Footnotes that work inside table floats	Ch. 6	—
`threeparttable`	Formal multi-note table blocks	Ch. 6 (optional)	—
`epstopdf`	Auto-converts legacy `.eps` figures	Ch. 7 (optional)	Needs shell-escape
`caption`	Caption font/label-separator customization	Ch. 7	Load alongside `subcaption`

Package	Purpose	First used	Load-order notes
`subcaption`	Subfigures (`subfigure` **package** is deprecated — don't use it)	Ch. 7	—
`hyperref`	Clickable cross-references and links	Ch. 8	Load near the **end** of the preamble
`cleveref`	`\cref`/`\Cref` — auto-prefixed references	Ch. 8	Load immediately **after** `hyperref`
`amsthm`	`\newtheorem`, `\theoremstyle`, `proof`	Ch. 11	—
`algorithm2e`	Pseudocode (`\KwIn`, `\For`, `\If`…)	Ch. 12	Don't load alongside `algorithm`/`algorithmic`
`algorithm`+`algorithmic`	Alternative pseudocode package pair	Ch. 12	Don't load alongside `algorithm2e`
`natbib`	`\citep`/`\citet` citation commands (BibTeX backend)	Ch. 13	—
`biblatex`	`\parencite`/`\textcite`, `\printbibliography` (Biber backend)	Ch. 13	Don't load alongside `natbib`
`url`	`\url{}` for clickable plain URLs	Ch. 13	—
`glossaries`	Acronyms, symbols lists, `\gls`	Ch. 16	—
`tikz`	Vector diagrams	Ch. 17 (elective)	—
`tikz-cd`	Commutative diagrams	Ch. 17 (elective)	—
`chemfig`	Chemical structure diagrams	Ch. 17 (elective)	—
`pgfplots`	Data plots from CSV/inline data	Ch. 18	—
`pgfplotstable`	Reading external `.csv` files into `pgfplots`	Ch. 18	—
`comment`	Hide a draft block with `\begin{comment}…\end{comment}`	Ch. 20	—
`geometry`	Margins, paper size	A.3 (this appendix)	Set once, early in the preamble

Golden load order, restated: most packages don't care about order, but two do — `hyperref` goes near the end of the preamble, `cleveref` immediately after it, and never load both `natbib` and `biblatex`, or both `algorithm2e` and `algorithm`/`algorithmic`, in the same document.

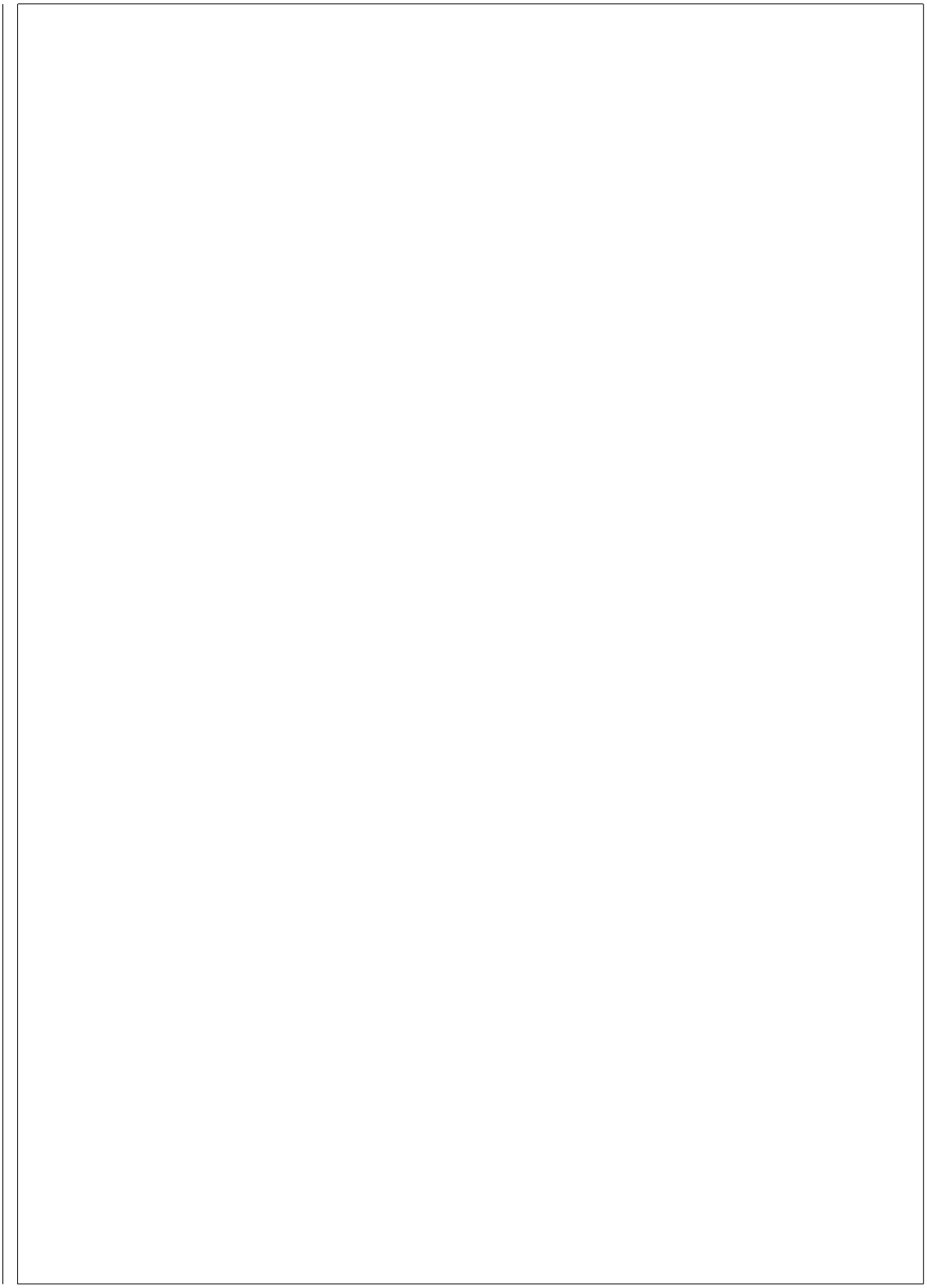




A.5 — Compiler Comparison

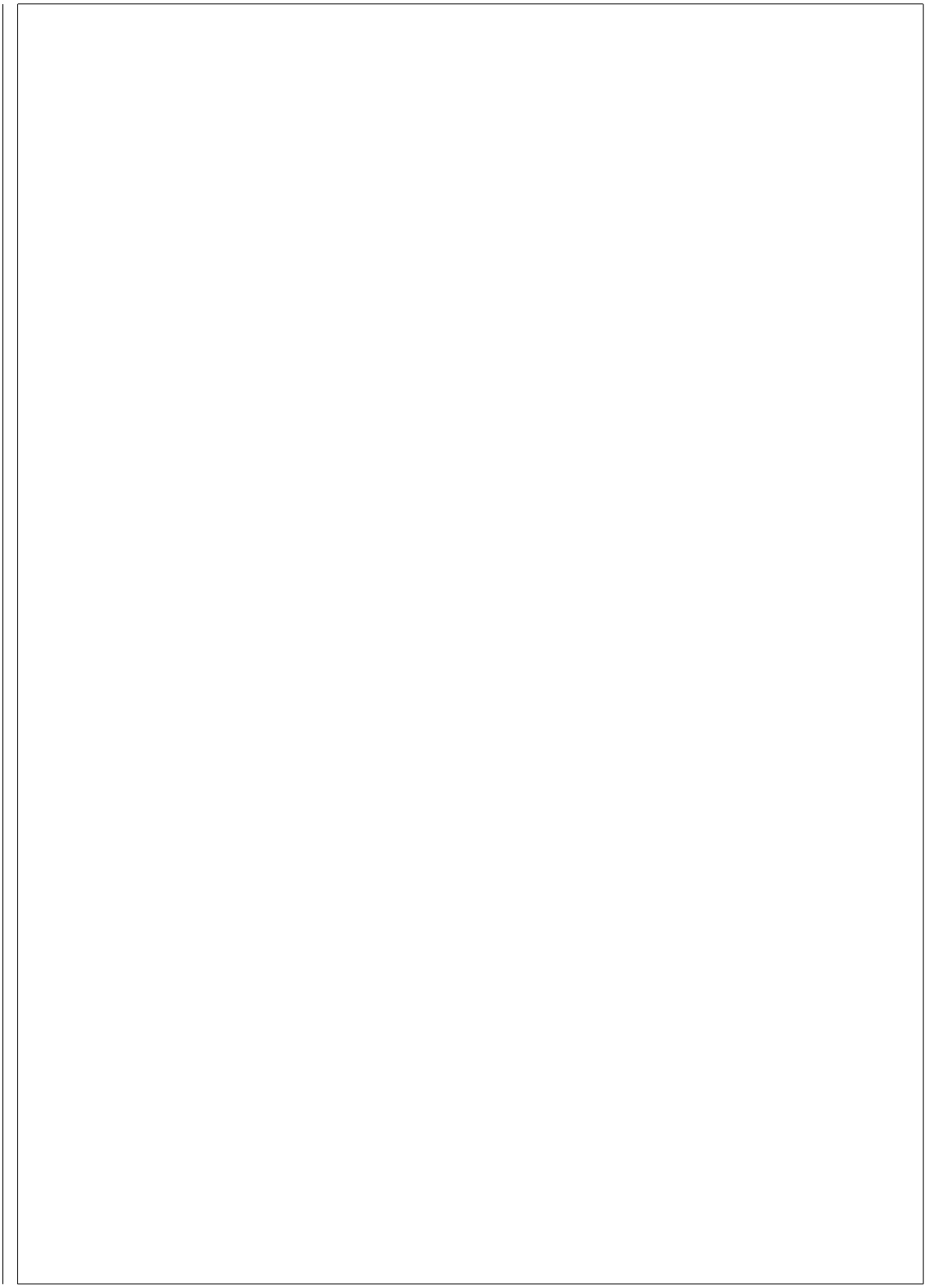
	pdfLaTeX	XeLaTeX	LuaLaTeX
Default in this guide	Yes	No	No
Custom fonts (`fontspec`)	Not supported	Supported	Supported
Unicode source files	Needs `inputenc` (Ch. 3)	Native	Native
Compile speed	Fastest	Slower	Slowest
Journal/template compatibility	Assumed by most packages and templates	Common for non-Latin scripts or custom typography	Common when Lua scripting is needed

Hard rule (first introduced in Ch. 2): if you use the `fontspec` package to change fonts, you *must* compile with XeLaTeX or LuaLaTeX — pdfLaTeX can't process it. If you don't need custom fonts, stick with pdfLaTeX; most journals require it for submission, and most packages assume it by default.



A.6 — `tikz-cd` and `chemfig` Starter Snippets

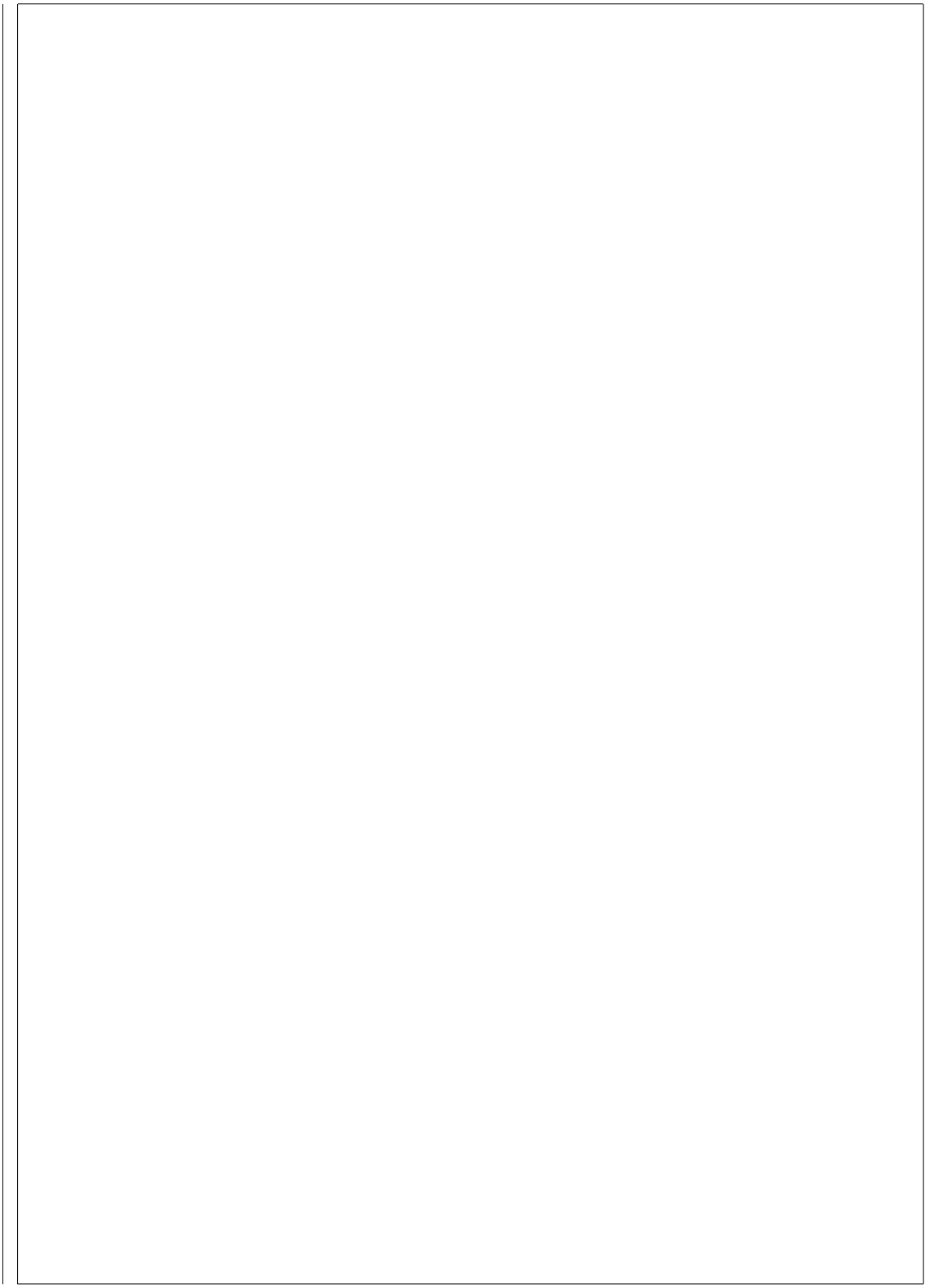
See Ch. 17's "Nuance: field-specific TikZ libraries" for both.



A.7 — Quick Troubleshooting Checklist

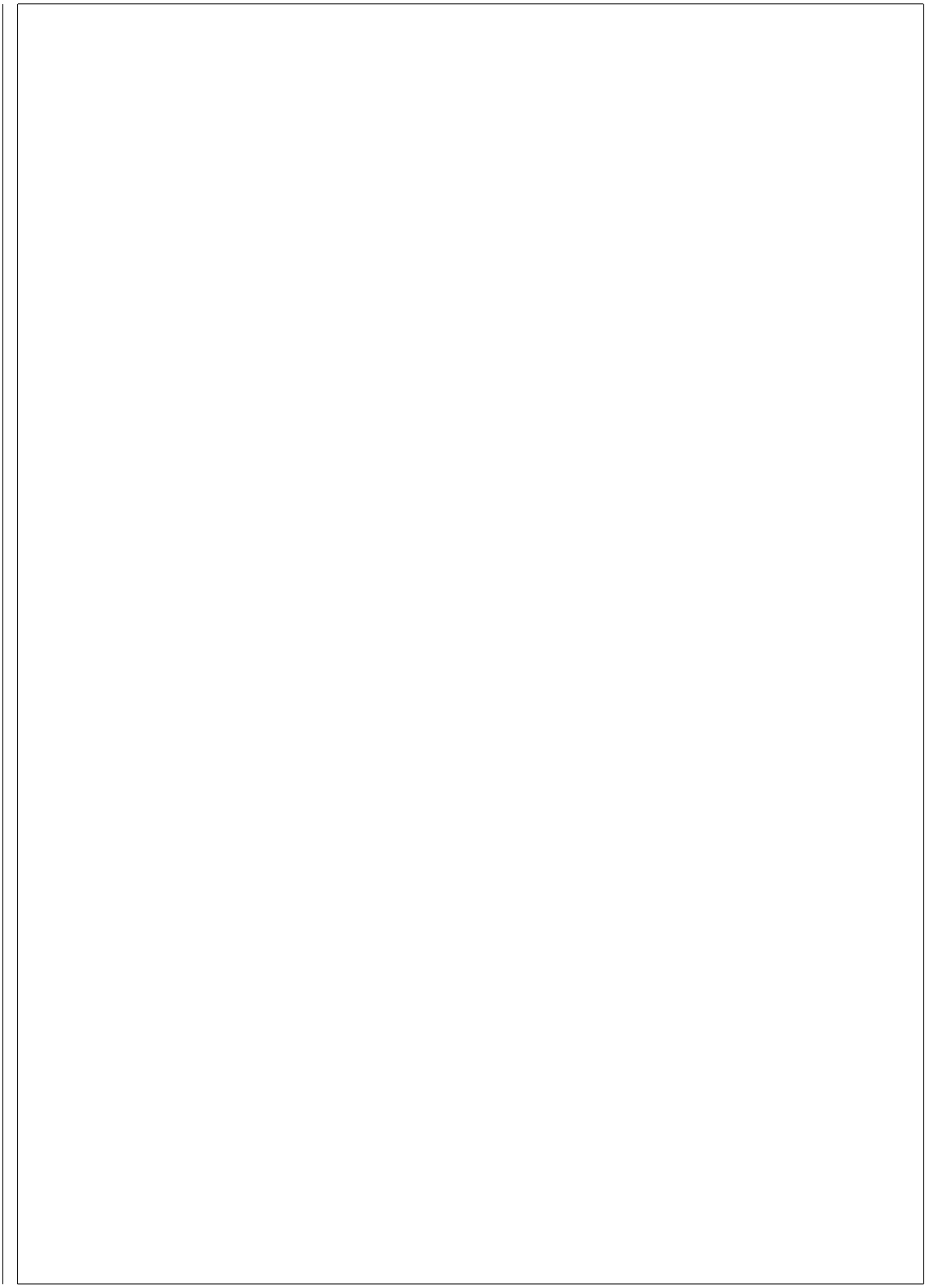
A fast first-pass reference — Ch. 22 (once drafted) will cover diagnostics in full depth. Until then:

- **Compile fails immediately** — check the *first* red error in the log, not the last one; with "Stop on first error" enabled (Ch. 2), this is usually the very next thing Overleaf shows you.
- **`Undefined control sequence`** — a typo'd command name, or a package that defines it was never loaded.
- **`Missing $ inserted`** — a math-only character (`_`, `^`) used outside math mode, or an unclosed `$`.
- **A strange error on the line *after* where the real mistake is** — classic symptom of an unclosed brace `{` on the previous line; LaTeX doesn't notice until it hits the next command (Ch. 2).
- **`??` in place of a number** — a `\label`/`\ref`/`\cref` mismatch; check the label spelling on both ends (Ch. 8).
- **A figure or table lands pages away from where you placed it** — LaTeX's float algorithm queued it; force it to flush with `\clearpage` (Ch. 7).
- **"No citation found" / empty bibliography** — almost always a compiler/backend mismatch: BibTeX for `\bibliography{}` (`natbib`), Biber for `\printbibliography` (`biblatex`) — check Overleaf's compiler settings (Ch. 13).
- **Two packages fighting each other** — don't load `natbib` and `biblatex` together, or `algorithm2e` and `algorithm`/`algorithmic` together (Ch. 12, Ch. 13).



A.8 — Starter `.bib` Files

See `supplementary_materials.md` → "Ch. 13 — Starter `.bib` Files for the 4 Capstone Topics."



A.9 — `hypersetup` Snippet

Ready to paste — professional link styling instead of the default colored boxes (full explanation in Ch. 8):

```
\hypersetup{
  colorlinks=true,
  linkcolor=blue,
  citecolor=blue,
  filecolor=blue,
  urlcolor=blue,
}
```

For a fully print-ready PDF with no visible link styling at all, use `hidelinks` instead of the block above.

End of Appendix.